

第五章

语法分析-自下而上分析

第五章

语法分析-自下而上分析

- 5.1 自下而上分析基本问题
- 5.2 算符优先分析
- 5.3 LR分析法
- 5.4 语法分析器的自动产生工具YACC

按照语法分析树的建立方法，把语法分析方法分成两类：

- 自上而下分析法

- 从文法的开始符号出发，向下推导，推出句子（输入串）。
- 从根结出发，自上而下地为输入串建立一棵语法树；

- 自下而上分析法

- 从输入串开始，逐步进行归约，直至归约到文法的开始符号。
- 归约：产生式的一个候选式（即右部符号）替换成该产生式的左部符号。
- 从语法树的末端开始，步步向上归约，直到根结

5.1 自下而上分析基本问题

5.1.1 归约

5.1.2 规范归约简述

5.1.3 符号栈的使用与语法树的表示

5.1.1 归约

- 采用“**移进—归约**”思想进行自下而上分析。
- 基本思想：用一个寄存符号的先进后出**栈**，把**输入符号**一个一个地移进到栈里，当**栈顶**形成某个产生式的候选式时，即把栈顶的这一部分替换成(**归约**为)该产生式的左部符号。

• 例：设文法5.1 $G(S)$:

(1) $S \rightarrow aAcBe$

(2) $A \rightarrow b$

(3) $A \rightarrow Ab$

(4) $B \rightarrow d$

试对abbcd进行“移进—归约”分析。

5.1.1 归约

- 自下而上分析的关键问题-可归约串

文法5.1 $G(S)$:

(1) $S \rightarrow aAcBe$

(2) $A \rightarrow b$

(3) $A \rightarrow Ab$

(4) $B \rightarrow d$

输入串:

abbcde

步骤:	1	2	3	4	5	6	7	8	9	10
动作:	进a	进b	归(2)	进b	归(3)	进c	进d	归(4)	进e	归(1)
									e	
							d	B	B	
				b		c	c	c	c	
	b	A	A	A	A	A	A	A	A	
a	a	a	a	a	a	a	a	a	a	S

图5.1 符号栈的变迁

5.1.1 归约

- 自下而上分析的关键问题-可归约串

文法5.1 $G(S)$:

(1) $S \rightarrow aAcBe$

(2) $A \rightarrow b$

(3) $A \rightarrow Ab$

(4) $B \rightarrow d$

步骤: 1 2 3 4 5
动作: 进a 进b 归(2) 进b 归(2)

			b	A					
	b	A	A	A					
a	a	a	a	a					

应用规则(2)

分析无法继续!

图5.1 符号栈的变迁

5.1.2 规范归约简述

- 短语: 文法 $G(S)$, 假定 $\alpha\beta\delta$ 是文法 G 的一个句型, 如果有

$$S \xRightarrow{*} \alpha A \delta \text{ 且 } A \xRightarrow{+} \beta$$

则称 β 是句型 $\alpha\beta\delta$ 相对于非终结符 A 的短语。

例: 设文法 $G(S)$:

- (1) $S \rightarrow aAcBe$
- (2) $A \rightarrow b$
- (3) $A \rightarrow Ab$
- (4) $B \rightarrow d$

$S \Rightarrow aAc\underline{B}e \Rightarrow a\underline{A}cde \Rightarrow a\underline{A}bcde$
 $S \Rightarrow a\underline{A}cBe \Rightarrow aAbc\underline{B}e \Rightarrow aAbcde$

句型 $aAbcde$ 的短语: $Ab, d, aAbcde$

5.1.2 规范归约简述

- 直接短语: 如果有 $A \Rightarrow \beta$, 则称 β 是句型 $\alpha\beta\delta$ 相对于规则 $A \rightarrow \beta$ 的直接短语。

例: 设文法 $G(S)$:

- (1) $S \rightarrow aAcBe$
- (2) $A \rightarrow b$
- (3) $A \rightarrow Ab$
- (4) $B \rightarrow d$

$S \Rightarrow aAc\underline{B}e \Rightarrow a\underline{A}cde \Rightarrow a\underline{A}bcde$
 $S \Rightarrow a\underline{A}cBe \Rightarrow aAbc\underline{B}e \Rightarrow$
 $aAbc\underline{d}e$

句型 $aAbcde$ 的短语: $Ab, d, aAbcde$

直接短语: Ab, d

5.1.2 规范归约简述

- 句柄: 一个句型的最左直接短语称为该句型的句柄。

例: 设文法 $G(S)$:

(1) $S \rightarrow aAcBe$

(2) $A \rightarrow b$

(3) $A \rightarrow Ab$

(4) $B \rightarrow d$

$S \Rightarrow aAc\underline{B}e \Rightarrow a\underline{A}cde \Rightarrow a\underline{Ab}cde$

$S \Rightarrow a\underline{A}cBe \Rightarrow aAbc\underline{B}e \Rightarrow$
 $aAbc\underline{d}e$

句型 $aAbcde$ 的短语: $Ab, d, aAbcde$

直接短语: Ab, d

句柄: Ab

5.1.2 规范归约简述

- 子树和短语(p86)
 - 子树：语法树的某个结点（作为子树的根）连同它的所有子孙组成。

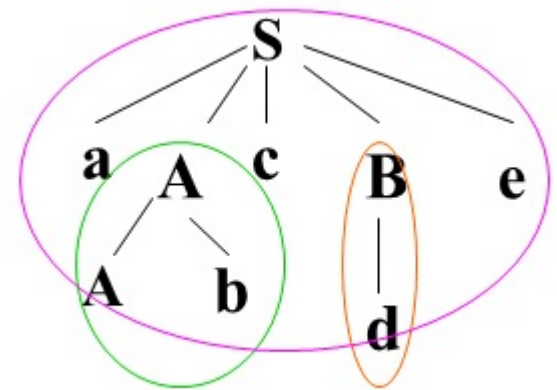
5.1.2 规范归约简述

- 子树和短语(p86)
 - **短语**: 一棵子树的所有端末结自左至右排列起来形成一个相对子树根的短语。
 - **直接短语**: 只有父子两代的子树形成的短语。
 - **句柄**: 句型语法树中最左那棵只有父子两代的子树形成的短语。

句型aAbcde的短语: Ab, d, aAbcde

直接短语: Ab, d

句柄: Ab



5.1.3 符号栈的使用与语法树的表示

文法(5.2) $G(E)$

$$E \rightarrow T \mid E + T$$
$$T \rightarrow F \mid T * F$$
$$F \rightarrow i \mid (E)$$

例5.1: 求句型 $i_1 * i_2 + i_3$ 的短语、直接短语和句柄。

例5.2: 求句型 $E + T * F + i$ 的短语、直接短语和句柄。

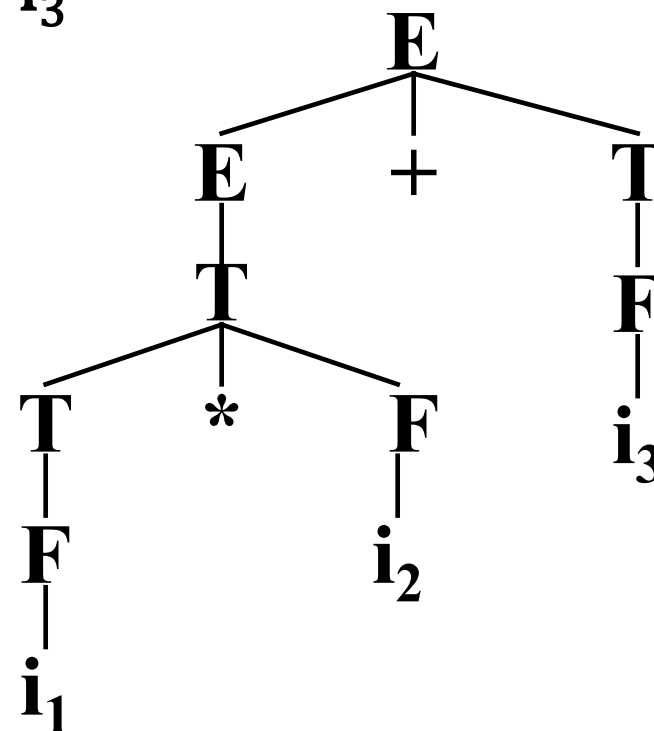
考虑文法 $G(E)$: $E \rightarrow T \mid E+T$

$T \rightarrow F \mid T * F$

$F \rightarrow (E) \mid i$

和句型 $i_1 * i_2 + i_3$:

- 短语: $i_1, i_2, i_3, i_1 * i_2, i_1 * i_2 + i_3$
- 直接短语: i_1, i_2, i_3
- 句柄: i_1



5.1.2 规范归约简述

- 用句柄归约和修剪语法树

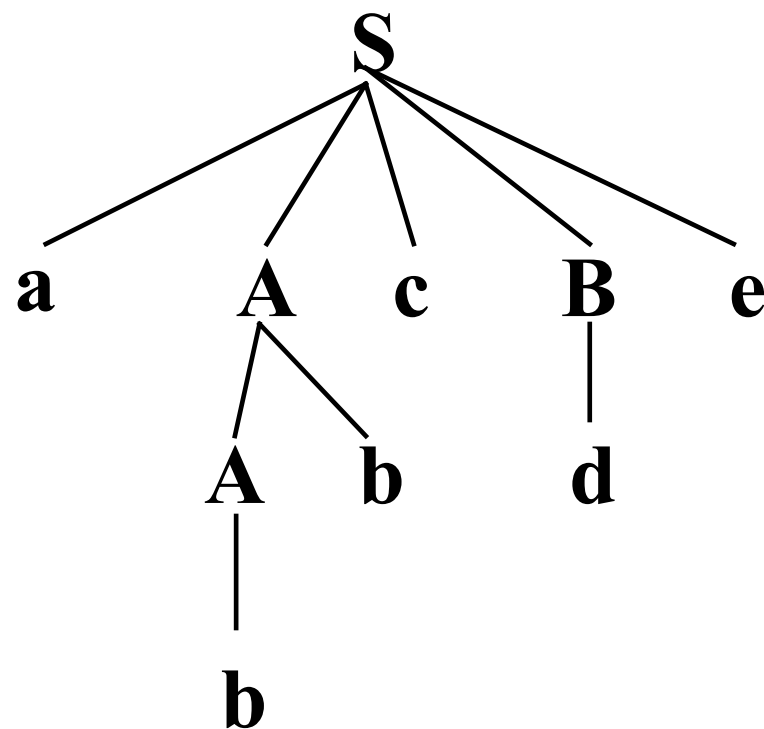
例：设文法 $G(S)$:

(1) $S \rightarrow aAcBe$

(2) $A \rightarrow b$

(3) $A \rightarrow Ab$

(4) $B \rightarrow d$



用句柄对句子归约。
自下而上分析：裁剪句柄。

- 定义：假定 α 是文法G的一个句子，我们称序列

$$\alpha_n, \alpha_{n-1}, \dots, \alpha_0$$

是 α 的一个规范归约，如果此序列满足：

1 $\alpha_n = \alpha$

2 α_0 为文法的开始符号，即 $\alpha_0 = S$

3 对任何 i ， $0 < i \leq n$ ， α_{i-1} 是从 α_i 经把句柄替换为相应产生式的左部符号而得到的。

5.1.2 规范归约简述

例：设文法 $G(S)$ ：

(1) $S \rightarrow aAcBe$

(2) $A \rightarrow b$

(3) $A \rightarrow Ab$

(4) $B \rightarrow d$

规范归约/最左归约

句型

句柄

归约规则

abbcde

b

(2) $A \rightarrow b$

a**A**bcde

Ab

(3) $A \rightarrow Ab$

a**A**cde

d

(4) $B \rightarrow d$

a**A**c**B**e

aAcBe

(1) $S \rightarrow aAcBe$

S

最右推导/
规范推导

$S \xRightarrow{(1)} aAcBe$

$\xRightarrow{(4)} aAcde$

$\xRightarrow{(3)} aAbcde$

$\xRightarrow{(2)} abbcde$

规范归约是最右推导的逆过程。

5.1.2 规范归约简述

- 由规范推导所得的句型称为规范句型。
- 对于规范句型，句柄的后面不会出现非终结符号。
- 用句柄刻画移进-归约过程的可归约串。
- 规范归约的实质：在移进过程中，当发现栈顶呈现句柄时，就用相应产生式的左部符号替换。

最右推导/
规范推导

$$\begin{aligned} S &\stackrel{(1)}{\Rightarrow} aAcBe \\ &\stackrel{(4)}{\Rightarrow} aAcde \\ &\stackrel{(3)}{\Rightarrow} aAbcde \\ &\stackrel{(2)}{\Rightarrow} abbcd e \end{aligned}$$

5.1.3 符号栈的使用与语法树的表示

- 符号栈的使用
 - 移进-归约分析器使用了一个符号栈和一个输入缓冲区。

一般形式:	符号栈的内容	剩余输入串
初态:	#	输入串#
终态:	#S	#

- 句型表示

符号栈内容 + 输入缓冲区内容 = # 当前句型 #

5.1.3 符号栈的使用与语法树的表示

- 分析程序的动作
 - 移进: 下一输入符号移进栈顶
 - 归约: 把句柄按产生式的左部进行归约
 - 接收: 分析程序报告成功
 - 出错: 发现了一个语法错,调用出错处理程序

注意：可归约串在栈顶, 不会在内部。

步骤	符号栈	输入串	动作
0	#	i1*i2+i3#	预备
1	#i1	*i2+i3#	进
2	#F	*i2+i3#	归, 用 $F \rightarrow i$
3	#T	*i2+i3#	归, 用 $T \rightarrow F$
4	#T*	i2+i3#	进
5	#T*i2	+i3#	进
6	#T*F	+i3#	归, 用 $F \rightarrow i$
7	#T	+i3#	归, 用 $T \rightarrow T*F$
8	#E	+i3#	归, 用 $E \rightarrow T$
9	#E+	i3#	进
10	#E+i3	#	进
11	#E+F	#	归, 用 $F \rightarrow i$
12	#E+T	#	归, 用 $T \rightarrow F$
13	#E	#	归, 用 $E \rightarrow E+T$
14	#E	#	接受

(1) $E \rightarrow T \mid E+T$

(2) $T \rightarrow F \mid T*F$

(3) $F \rightarrow i \mid (E)$

自下而上分析

- 采用“**移进—归约**”思想进行自下而上分析。
- 基本思想：
用一个寄存符号的先进后出**栈**，把**输入符号**一个一个地移进到栈里，当**栈顶**形成某个产生式的候选式时，即把栈顶的这一部分替换成(**归约**为)该产生式的左部符号。

如何在句型中找出句柄？

5.2 算符优先分析

- 算符优先分析法: 定义算符(终结符)之间的某种优先关系, 借助于这种关系寻找“可归约串”和进行归约。

Robert W. Floyd
1978年的图灵奖获得者



“Syntactic analysis and operator precedence,” *Journal of the Association for Computing Machinery* **10** (1963), 316-333.

5.2 算符优先分析

- 算符优先关系

两个终结符a与b的优先关系

$a \lessdot b$ a的优先级低于b

$a \gtrdot b$ a的优先级高于b

$a \doteq b$ a的优先级等于b

- 注意：与数学上的 $< > =$ 不同

- $+ \gtrdot +$ 和 $= \lessdot =$

- $a \lessdot b$ 并不意味着 $b \lessdot a$ ，如 $(\lessdot +$ 和 $+ \lessdot ($

- $a \doteq b$ 也不一定意味着 $b \doteq a$ ，如 $(\doteq)$

5.2.1 算符优先文法及优先表构造

- 算符文法

一个文法，如果它的任一产生式的右部都**不含两个相继(并列)的非终结符**，即不含如下形式的产生式右部：

...QR...

则我们称该文法为算符文法。

文法G(E):

$E \rightarrow EAE \mid (E) \mid i$ 

$A \rightarrow + \mid *$

文法G(E):

$E \rightarrow E+E \mid E * E \mid (E) \mid i$ 

5.2.1 算符优先文法及优先表构造

- 约定：
 - a、b代表任意终结符；
 - P、Q、R代表任意非终结符；
 - ‘...’ 代表由终结符和非终结符组成的任意序列，包括空字。

5.2.1 算符优先文法及优先表构造

- 算符文法

- 算符文法任一产生式的右部具有如下形式:

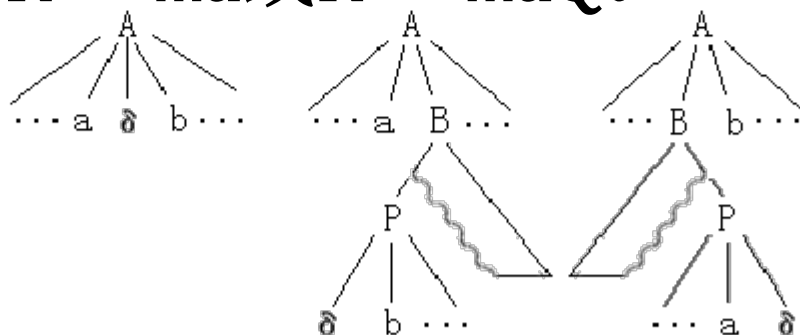
$$N_1 a_1 N_2 a_2 \dots N_n a_n N_{n+1}$$

其中, a_i 是终结符, N_i 是可有可无的非终结符。

算符文法

– 假定G是一个不含 ε -产生式的算符文法。对于任何一对终结符a、b:

- $a \equiv b$: 当且仅当文法G中含有形如 $P \rightarrow \dots ab \dots$ 或 $P \rightarrow \dots aQb \dots$ 的产生式;
- $a < b$: 当且仅当G中含有形如 $P \rightarrow \dots aR \dots$ 的产生式, 而 $R \xRightarrow{+} b \dots$ 或 $R \xRightarrow{+} Qb \dots$;
- $a > b$: 当且仅当G中含有形如 $P \rightarrow \dots Rb \dots$ 的产生式, 而 $R \xRightarrow{+} \dots a$ 或 $R \xRightarrow{+} \dots aQ$ 。



(a) $a \equiv b$

(b) $a < b$

(c) $a > b$

5.2.1 算符优先文法及优先表构造

- 算符优先文法

如果一个算符文法G中的任何终结符对(a, b)至多只满足下述三关系之一:

$a \doteq b$, $a \leq b$, $a \geq b$

则称G是一个算符优先文法。

5.2.1 算符优先文法及优先表构造

- 算符优先文法

文法G(E):

$E \rightarrow E + E \mid E * E \mid (E) \mid i$



是否为算符优先文法?

根据规则(3):

$E \rightarrow E + E$

Rb...

$E \Rightarrow E + E$

aQ

则: $a > b$

即: $+ > +$

根据规则(2):

$E \rightarrow E + E$

aR...

$E \Rightarrow E + E$

Qb

则: $a < b$

即: $+ < +$

5.2.1 算符优先文法及优先表构造

- 例5.4: 文法5.3 $G(E)$:

$$(1) E \rightarrow E + T \mid T$$

$$(2) T \rightarrow T * F \mid F$$

$$(3) F \rightarrow P \uparrow F \mid P$$

$$(4) P \rightarrow (E) \mid i$$

$$E \rightarrow E + T \text{ 且 } E \Rightarrow E + T$$

$$+ \succ +$$

$$E \rightarrow E + T \text{ 且 } T \Rightarrow T * F$$

$$+ \prec *$$

$$E \rightarrow E + T \text{ 且 } T \overset{+}{\Rightarrow} (E)$$

$$+ \prec ($$

$$P \rightarrow (E) \text{ 且 } E \Rightarrow E + T$$

$$(\prec +$$

$$P \rightarrow (E)$$

$$(\doteq)$$

$$F \rightarrow P \uparrow F \text{ 且 } F \Rightarrow P \uparrow F$$

$$\uparrow \prec \uparrow$$

5.2.1 算符优先文法及优先表构造

- 例5.4: 文法5.3 $G(E)$:

(1) $E \rightarrow E + T \mid T$

(2) $T \rightarrow T * F \mid F$

(3) $F \rightarrow P \uparrow F \mid P$

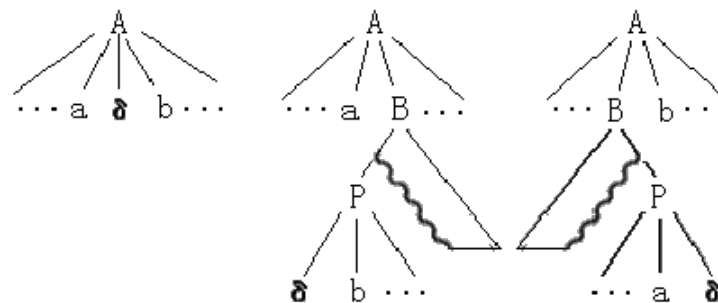
(4) $P \rightarrow (E) \mid i$

	+	*	$\uparrow$	i	(	)	#
+	$\succ$	$\prec$	$\prec$	$\prec$	$\prec$	$\succ$	$\succ$
*	$\succ$	$\succ$	$\prec$	$\prec$	$\prec$	$\succ$	$\succ$
$\uparrow$	$\succ$	$\succ$	$\prec$	$\prec$	$\prec$	$\succ$	$\succ$
i	$\succ$	$\succ$	$\succ$			$\succ$	$\succ$
(	$\prec$	$\prec$	$\prec$	$\prec$	$\prec$	$\doteq$	
)	$\succ$	$\succ$	$\succ$			$\succ$	$\succ$
#	$\prec$	$\prec$	$\prec$	$\prec$	$\prec$		

5.2.1 算符优先文法及优先表构造

- 算符优先关系

- $a \doteq b$: 当且仅当文法G中含有形如 $P \rightarrow \dots ab \dots$ 或 $P \rightarrow \dots aQb \dots$ 的产生式; 检查产生式的候选式
- $a < b$: 当且仅当G中含有形如 $P \rightarrow \dots aR \dots$ 的产生式, 而 $R \xRightarrow{+} b \dots$ 或 $R \xRightarrow{+} Qb \dots$; 构造R的FIRSTVT集合
- $a > b$: 当且仅当G中含有形如 $P \rightarrow \dots Rb \dots$ 的产生式, 而 $R \xRightarrow{+} \dots a$ 或 $R \xRightarrow{+} \dots aQ$; 构造R的LASTVT集合



(a) $a \doteq b$

(b) $a < b$

(c) $a > b$

5.2.1 算符优先文法及优先表构造

- FIRSTVT

$$FIRSTVT(P) = \{a \mid P \overset{+}{\Rightarrow} a \cdots, \text{或} P \overset{+}{\Rightarrow} Qa \cdots, a \in V_T \text{而} Q \in V_N\}$$

$$FIRSTVT(P) = \{a \mid P \overset{+}{\Rightarrow} a \cdots, \text{ 或 } P \overset{+}{\Rightarrow} Qa \cdots, a \in V_T \text{ 而 } Q \in V_N\}$$

- 构造集合FIRSTVT(P)的算法。
- 按其定义，可用下面两条规则来构造集合FIRSTVT(P):
 1. 若有产生式 $P \rightarrow a \dots$ 或 $P \rightarrow Qa \dots$ ，则
 $a \in FIRSTVT(P)$;
 2. 若 $a \in FIRSTVT(Q)$ ，且有产生式 $P \rightarrow Q \dots$ ，则
 $a \in FIRSTVT(P)$ 。

- 数据结构：
 - 布尔数组 $F[P, a]$ ，使得 $F[P, a]$ 为真的条件是，当且仅当 $a \in \text{FIRSTVT}(P)$ 。开始时，按上述的规则(1)对每个数组元素 $F[P, a]$ 赋初值。
 - 栈STACK，把所有初值为真的数组元素 $F[P, a]$ 的符号对 (P, a) 全都放在STACK之中。

- 运算:

- 如果栈STACK不空, 就将顶项逐出, 记此项为(Q, a)。对于每个形如

$$P \rightarrow Q \dots$$

的产生式, 若F[P, a]为假, 则变其值为真且将(P, a)推进STACK栈。

- 上述过程必须一直重复, 直至栈STACK拆空为止。

- 程序:

```
PROCEDURE INSERT(P, a);  
IF NOT F[P, a] THEN  
BEGIN  
    F[P, a]:=TRUE;  
    把(P, a)下推进STACK栈  
END;
```

主程序:

BEGIN

FOR 每个非终结符 P 和终结符 a DO

$F[P, a] := \text{FALSE};$

FOR 每个形如 $P \rightarrow a \dots$ 或 $P \rightarrow Qa \dots$ 的产生式 DO

INSERT(P, a);

WHILE STACK 非空 DO

BEGIN

把STACK的顶项, 记为 (Q, a) , 上托出去;

FOR 每条形如 $P \rightarrow Q \dots$ 的产生式 DO

INSERT(P, a);

END OF WHILE;

END

	+	*	↑	i	(	)
E	✓	✓	✓	✓	✓	
T		✓	✓	✓	✓	
F			✓	✓	✓	
P				✓	✓	

例：文法5.3 $G(E)$:

(1) $E \rightarrow E + T \mid T$

(2) $T \rightarrow T * F \mid F$

(3) $F \rightarrow P \uparrow F \mid P$

(4) $P \rightarrow (E) \mid i$

$$FIRSTVT(T) = \{*, \uparrow, (, i\}$$

$$FIRSTVT(F) = \{\uparrow, (, i\}$$

$$FIRSTVT(E) = \{+, *, \uparrow, (, i\}$$

$$FIRSTVT(P) = \{(, i\}$$

5.2.1 算符优先文法及优先表构造

- LASTVT

$$LASTVT(P) = \{a \mid P \overset{+}{\Rightarrow} \cdots a, \text{或} P \overset{+}{\Rightarrow} \cdots aQ, a \in V_T \text{而} Q \in V_N\}$$

$$LASTVT(P) = \{a \mid P \xRightarrow{+} \cdots a, \text{或} P \xRightarrow{+} \cdots aQ, a \in V_T \text{而} Q \in V_N\}$$

- 构造集合LASTVT(P)的算法。
- 按其定义，可用下面两条规则来构造集合LASTVT(P):
 1. 若有产生式 $P \rightarrow \dots a$ 或 $P \rightarrow \dots aQ$ ，则 $a \in LASTVT(P)$;
 2. 若 $a \in LASTVT(Q)$ ，且有产生式 $P \rightarrow \dots Q$ ，则 $a \in LASTVT(P)$ 。

- 数据结构：
 - 布尔数组 $F[P, a]$ ，使得 $F[P, a]$ 为真的条件是，当且仅当 $a \in \text{LASTVT}(P)$ 。开始时，按上述的规则(1)对每个数组元素 $F[P, a]$ 赋初值。
 - 栈STACK，把所有初值为真的数组元素 $F[P, a]$ 的符号对 (P, a) 全都放在STACK之中。
- 运算：
 - 如果栈STACK不空，就将顶项逐出，记此项为 (Q, a) 。对于每个形如

$$P \rightarrow \dots Q$$
 的产生式，若 $F[P, a]$ 为假，则变其值为真且将 (P, a) 推进STACK栈。
 - 上述过程必须一直重复，直至栈STACK拆空为止。

- 程序:

```
PROCEDURE INSERT(P, a);  
IF NOT F[P, a] THEN  
BEGIN  
    F[P, a]:=TRUE;  
    把(P, a)下推进STACK栈  
END;
```

主程序:

BEGIN

FOR 每个非终结符 P 和终结符 a DO

$F[P, a] := \text{FALSE};$

FOR 每个形如 $P \rightarrow \dots a$ 或 $P \rightarrow \dots aQ$ 的产生式 DO

INSERT(P, a);

WHILE STACK 非空 DO

BEGIN

把STACK的顶项, 记为 (Q, a) , 上托出去;

FOR 每条形如 $P \rightarrow \dots Q$ 的产生式 DO

INSERT(P, a);

END OF WHILE;

END

例：文法5.3 $G(E)$:

(1) $E \rightarrow E + T \mid T$

(2) $T \rightarrow T * F \mid F$

(3) $F \rightarrow P \uparrow F \mid P$

(4) $P \rightarrow (E) \mid i$

	+	*	$\uparrow$	i	(	)
E	✓	✓	✓	✓		✓
T		✓	✓	✓		✓
F			✓	✓		✓
P				✓		✓

$$LASTVT(T) = \{*, \uparrow,), i\}$$

$$LASTVT(F) = \{\uparrow,), i\}$$

$$LASTVT(E) = \{+, *, \uparrow,), i\}$$

$$LASTVT(P) = \{), i\}$$

	+	*	↑	i	(	)
E	√	√	√	√	√	
T		√	√	√	√	
F			√	√	√	
P				√	√	

	+	*	↑	i	(	)
E	√	√	√	√		√
T		√	√	√		√
F			√	√		√
P				√		√

$$FIRSTVT(T) = \{*, \uparrow, (, i\}$$

$$FIRSTVT(F) = \{\uparrow, (, i\}$$

$$FIRSTVT(E) = \{+, *, \uparrow, (, i\}$$

$$FIRSTVT(P) = \{(, i\}$$

$$LASTVT(T) = \{*, \uparrow,), i\}$$

$$LASTVT(F) = \{\uparrow,), i\}$$

$$LASTVT(E) = \{+, *, \uparrow,), i\}$$

$$LASTVT(P) = \{), i\}$$

5.2.1 算符优先文法及优先表构造

- 构造算符优先表

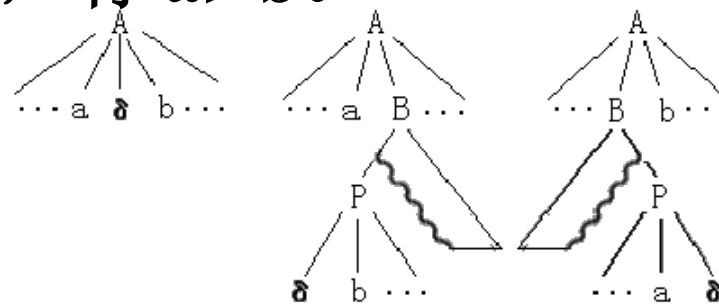
- $a \doteq b$

- $a \lessdot b$: 假定有个产生式的一个候选形为
 $\dots aP\dots$

对任何 $b \in \text{FIRSTVT}(P)$, 有 $a \lessdot b$ 。

- $a \gtrdot b$: 假定有个产生式的一个候选形为
 $\dots Pb\dots$

对任何 $a \in \text{LASTVT}(P)$, 有 $a \gtrdot b$ 。



(a) $a \doteq b$

(b) $a \lessdot b$

(c) $a \gtrdot b$

FOR 每个产生式 $P \rightarrow X_1X_2...X_n$ DO

FOR $i:=1$ TO $n-1$ DO

BEGIN

IF X_i 和 X_{i+1} 均为终结符 THEN 置 $X_i \doteq X_{i+1}$

IF $i \leq n-2$ 且 X_i 和 X_{i+2} 都为终结符

但 X_{i+1} 为非终结符 THEN 置 $X_i \doteq X_{i+2}$;

IF X_i 为终结符而 X_{i+1} 为非终结符 THEN

FOR FIRSTVT(X_{i+1})中的每个 a DO

置 $X_i \triangleleft a$;

IF X_i 为非终结符而 X_{i+1} 为终结符 THEN

FOR LASTVT(X_i)中的每个 a DO

置 $a \triangleright X_{i+1}$

END

- 例：考虑下面的文法G(E)：

(1) $E \rightarrow E + T \mid T$

(2) $T \rightarrow T * F \mid F$

(3) $F \rightarrow P \uparrow F \mid P$

(4) $P \rightarrow (E) \mid i$

1. 计算文法G的FIRSTVT和LASTVT;
2. 构造优先关系表。

FIRSTVT

	+	*	↑	i	(	)
E	√	√	√	√	√	
T		√	√	√	√	
F			√	√	√	
P				√	√	

LASTVT

	+	*	↑	i	(	)
E	√	√	√	√		√
T		√	√	√		√
F			√	√		√
P				√		√

例：文法5.3 $G(E)$:

(1) $E \rightarrow E + T \mid T$

(2) $T \rightarrow T * F \mid F$

(3) $F \rightarrow P \uparrow F \mid P$

(4) $P \rightarrow (E) \mid i$

FOR 每个产生式 $P \rightarrow X_1 X_2 \dots X_n$ DO

FOR $i := 1$ TO $n-1$ DO

BEGIN

IF X_i 和 X_{i+1} 均为终结符 THEN 置 $X_i \doteq X_{i+1}$

IF $i \leq n-2$ 且 X_i 和 X_{i+2} 都为终结符

但 X_{i+1} 为非终结符 THEN 置 $X_i \doteq X_{i+2}$;

IF X_i 为终结符而 X_{i+1} 为非终结符 THEN

FOR FIRSTVT(X_{i+1}) 中的每个 a DO

置 $X_i < a$;

IF X_i 为非终结符而 X_{i+1} 为终结符 THEN

FOR LASTVT(X_i) 中的每个 a DO

置 $a > X_{i+1}$

END

□ G的算符优先关系表

	+	*	↑	i	(	)	#
+	⋈	⋈	⋈	⋈	⋈	⋈	⋈
*	⋈	⋈	⋈	⋈	⋈	⋈	⋈
↑	⋈	⋈	⋈	⋈	⋈	⋈	⋈
i	⋈	⋈	⋈			⋈	⋈
(	⋈	⋈	⋈	⋈	⋈	≡	
)	⋈	⋈	⋈			⋈	⋈
#	⋈	⋈	⋈	⋈	⋈		

5.2.2 算符优先分析算法

- 短语: 文法 $G(S)$, 假定 $\alpha\beta\delta$ 是文法 G 的一个句型, 如果有

$$S \overset{*}{\Rightarrow} \alpha A \delta \text{ 且 } A \overset{+}{\Rightarrow} \beta$$

则称 β 是句型 $\alpha\beta\delta$ 相对于非终结符 A 的短语。

例: 文法5.3 $G(E)$:

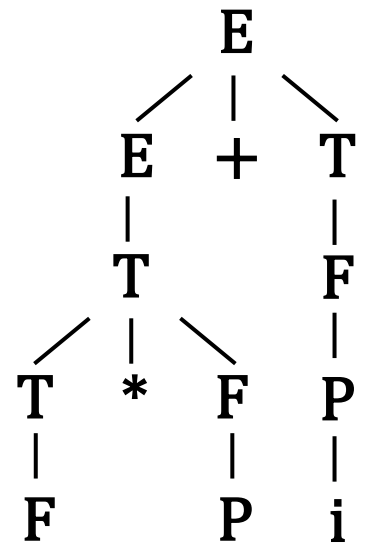
$$(1) E \rightarrow E + T \mid T$$

$$(2) T \rightarrow T * F \mid F$$

$$(3) F \rightarrow P \uparrow F \mid P$$

$$(4) P \rightarrow (E) \mid i$$

句型 $F * P + i$ 的短语: $F, P, F * P, i, F * P + i$



5.2.2 算符优先分析算法

- **素短语**: 是指一个短语, 它至少含有一个终结符, 并且, 除它自身之外不再含任何更小的素短语。
- **最左素短语**: 处于句型最左边的素短语。

例: 文法5.3 $G(E)$:

(1) $E \rightarrow E + T \mid T$

(2) $T \rightarrow T * F \mid F$

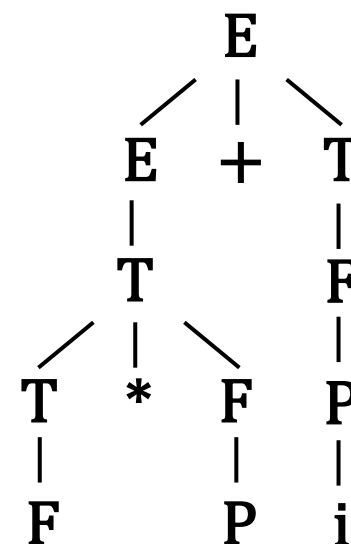
(3) $F \rightarrow P \uparrow F \mid P$

(4) $P \rightarrow (E) \mid i$

句型 $F * P + i$ 的短语: $F, P, F * P, i, F * P + i$

素短语: $F * P, i$

最左素短语: $F * P$



- 算符优先文法句型(括在两个#之间)的一般形式写成:

$$\#N_1a_1N_2a_2\ldots N_na_nN_{n+1}\#$$

其中, 每个 a_i 都是终结符, N_i 是可有可无的非终结符。

- 定理: 一个算符优先文法 G 的任何句型的最左素短语是满足如下条件的最左子串 $N_ja_j\ldots N_ia_iN_{i+1}$,

$$a_{j-1} < a_j$$

$$a_j \doteq a_{j+1}, \quad \ldots, \quad a_{i-1} \doteq a_i$$

$$a_i > a_{i+1}$$

- 用任意名字如N代替归约的非终结符;
- 最左素短语 $N_1a_1...N_ia_iN_{i+1}$ 归约到一个非终结符N:
N: 产生式的左部符号,
该产生式的右部和 $N_1a_1...N_ia_iN_{i+1}$ 自左至右一一对应, 终结符对终结符, 非终结符对非终结符, 对应的终结符相同。

输入串: i_1+i_2

栈	关系	串	动作
#	$\lessdot$	$i_1+i_2\#$	
# $\lessdot i_1$	$\gtrdot$	$+i_2\#$	移进
#N	$\lessdot$	$+i_2\#$	归约
# $\lessdot$ N+	$\lessdot$	$i_2\#$	移进
# $\lessdot$ N+ $\lessdot i_2$	$\gtrdot$	#	移进
# $\lessdot$ N+N	$\gtrdot$	#	归约
#N		#	接受

文法5.3 $G(E)$:

(1) $E \rightarrow E+T \mid T$

(2) $T \rightarrow T * F \mid F$

(3) $F \rightarrow P \uparrow F \mid P$

(4) $P \rightarrow (E) \mid i$

输入符号

	+	*	$\uparrow$	i	(	)	#
+	$\gtrdot$	$\lessdot$	$\lessdot$	$\lessdot$	$\lessdot$	$\gtrdot$	$\gtrdot$
*	$\gtrdot$	$\gtrdot$	$\lessdot$	$\lessdot$	$\lessdot$	$\gtrdot$	$\gtrdot$
$\uparrow$	$\gtrdot$	$\gtrdot$	$\lessdot$	$\lessdot$	$\lessdot$	$\gtrdot$	$\gtrdot$
i	$\gtrdot$	$\gtrdot$	$\gtrdot$			$\gtrdot$	$\gtrdot$
(	$\lessdot$	$\lessdot$	$\lessdot$	$\lessdot$	$\lessdot$	$\doteq$	
)	$\gtrdot$	$\gtrdot$	$\gtrdot$			$\gtrdot$	$\gtrdot$
#	$\lessdot$	$\lessdot$	$\lessdot$	$\lessdot$	$\lessdot$		

栈内符号

5.2.2 算符优先分析算法

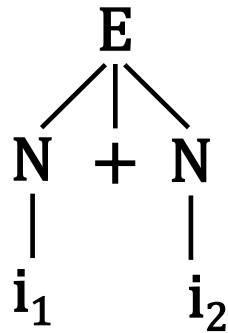
- 用符号栈S实现最左素短语的查找
 1. 将输入串依次逐个存入符号栈S中，直到符号栈顶元素与下一个待输入的符号有优先关系 $\succ$ 为止，栈顶即为最左素短语的右端；
 2. 在栈内往栈底方向找第一个 $\lessdot$ 关系(相邻两个终结符)，即寻找最左素短语的左端；
 3. 将 $\lessdot$ 和 $\succ$ 之间的符号串(包括中间的和两边的非终结符)弹出栈，并将归约后的非终结符压入栈，完成一次归约。

```

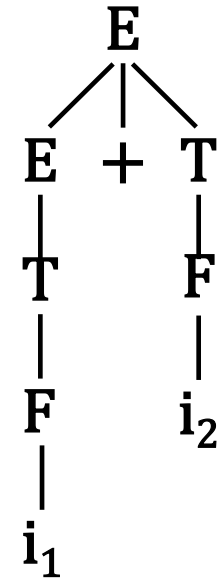
k:=1; S[k]:= ‘#’ ;                                /*栈顶（文法符号）*/
REPEAT
    把下一个输入符号读进a中;
    IF S[k]∈VT THEN j:=k ELSE j:=k-1;          /*栈顶j-终结符*/
    WHILE S[j]>a DO                                /*可归约串右端*/
    BEGIN
        REPEAT
            Q:=S[j];
            IF S[j-1]∈VT THEN j:=j-1 ELSE j:=j-2
        UNTIL S[j]<Q;                                /*可归约串左端*/
        k:=j+1;                                       /*把S[j+1]...S[k]归约为某个N*/
        S[k]:=N
    END OF WHILE;
    IF S[j]<a OR S[j]≠a THEN
        BEGIN k:=k+1; S[k]:=a END                    /*输入字符进栈*/
    ELSE ERROR /*调用出错诊察程序*/
UNTIL a= ‘#’

```

算符优先分析：语法架子树



规范归约：语法树



算符优先分析跳过了所有单非产生式所对应的归约步骤。

- 算符优先分析法特点：
 - 优点：简单，快速
 - 缺点：可能错误接受非法句子，适用于表达式的语法分析。

5.2.3 优先函数

- 把每个终结符 θ 与两个自然数 $f(\theta)$ 与 $g(\theta)$ 相对应, 使得
 - 若 $\theta_1 \triangleleft \theta_2$, 则 $f(\theta_1) < g(\theta_2)$
 - 若 $\theta_1 \doteq \theta_2$, 则 $f(\theta_1) = g(\theta_2)$
 - 若 $\theta_1 \triangleright \theta_2$, 则 $f(\theta_1) > g(\theta_2)$

f 称为入栈优先函数, g 称为比较优先函数。
- 优点: 便于比较, 节省空间;
- 缺点: 原来不存在优先关系的两个终结符, 由于自然数相对应, 变成可以比较的。要进行一些特殊的判断。

- 文法G(E)

$$(1) E \rightarrow E + T \mid T$$

$$(2) T \rightarrow T * F \mid F$$

$$(3) F \rightarrow P \uparrow F \mid P$$

$$(4) P \rightarrow (E) \mid i$$

优先函数如下表

	+	*	↑	i	(	)	#
+	⋈	⋈	⋈	⋈	⋈	⋈	⋈
*	⋈	⋈	⋈	⋈	⋈	⋈	⋈
↑	⋈	⋈	⋈	⋈	⋈	⋈	⋈
i	⋈	⋈	⋈			⋈	⋈
(	⋈	⋈	⋈	⋈	⋈	⋈	
)	⋈	⋈	⋈			⋈	⋈
#	⋈	⋈	⋈	⋈	⋈		

	+	*	↑	(	)	i	#
F	2	4	4	0	6	6	0
G	1	3	5	5	0	5	0

- 从优先表构造优先函数:
 - 1) 对于每个终结符 a ，令其对应两个符号 f_a 和 g_a ，画一张以所有符号为结点的方向图。如果 $a \succ b$ 或 $a \doteq b$ ，则从 f_a 画一条弧至 g_b ，如果 $a \prec b$ 或 $a \doteq b$ ，则画一条弧从 g_b 至 f_a 。
 - 2) 对每个结点都赋予一个数，此数等于从该结点出发所能到达的结点(包括出发点自身)的个数。赋给 f_a 的数作为 $f(a)$ ，赋给 g_a 的数作为 $g(a)$ 。
 - 3) 检查所构造出来的函数 f 和 g 是否与原来的关系矛盾。若没有矛盾，则 f 和 g 就是要求的优先函数，若有矛盾，则不存在优先函数。

• 例: 文法G(E)

(1) $E \rightarrow E + T \mid T$

(2) $T \rightarrow T * F \mid F$

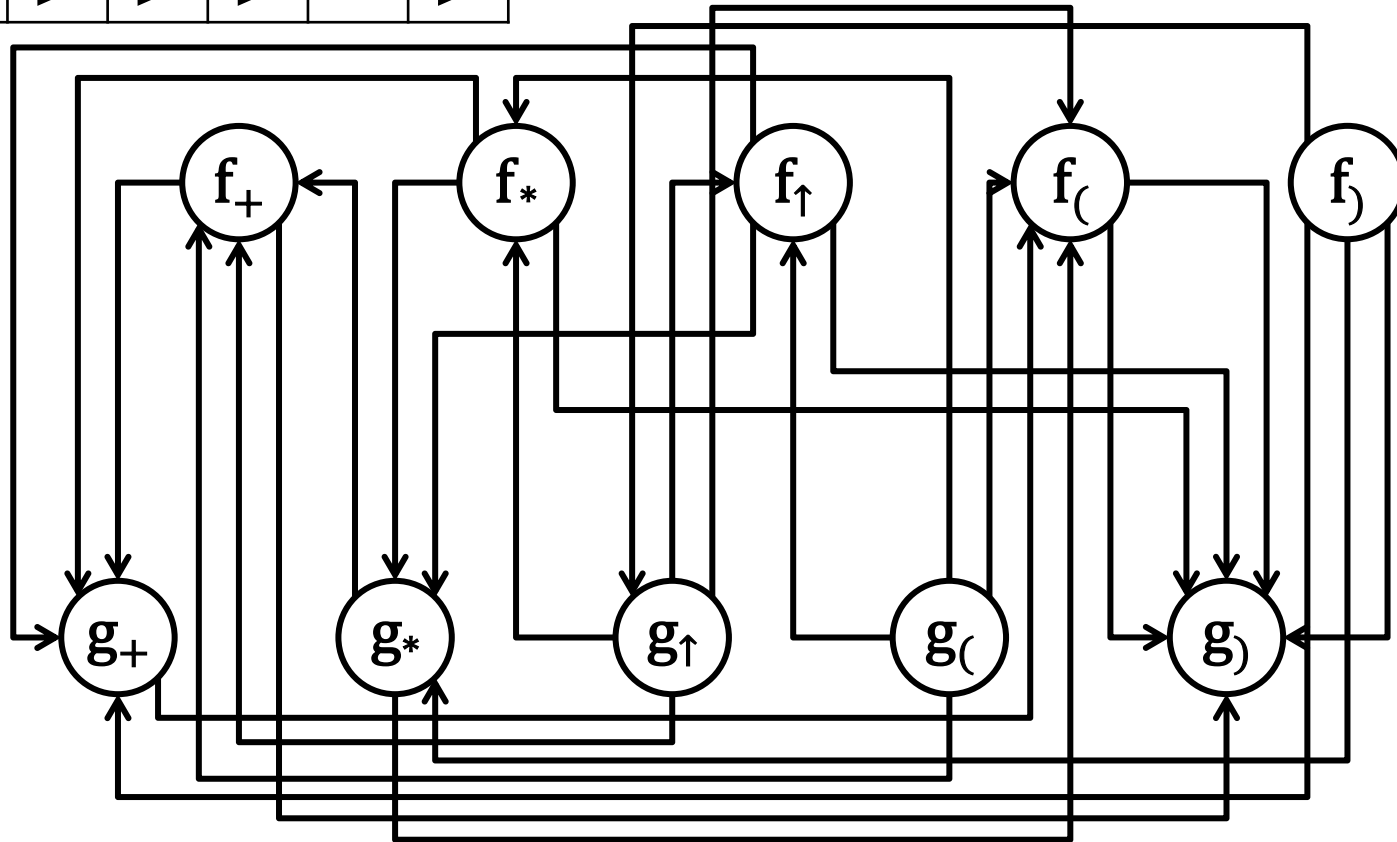
(3) $F \rightarrow P \uparrow F \mid P$

(4) $P \rightarrow (E) \mid i$

终结符 $+$, $*$, $\uparrow$, $($, $)$

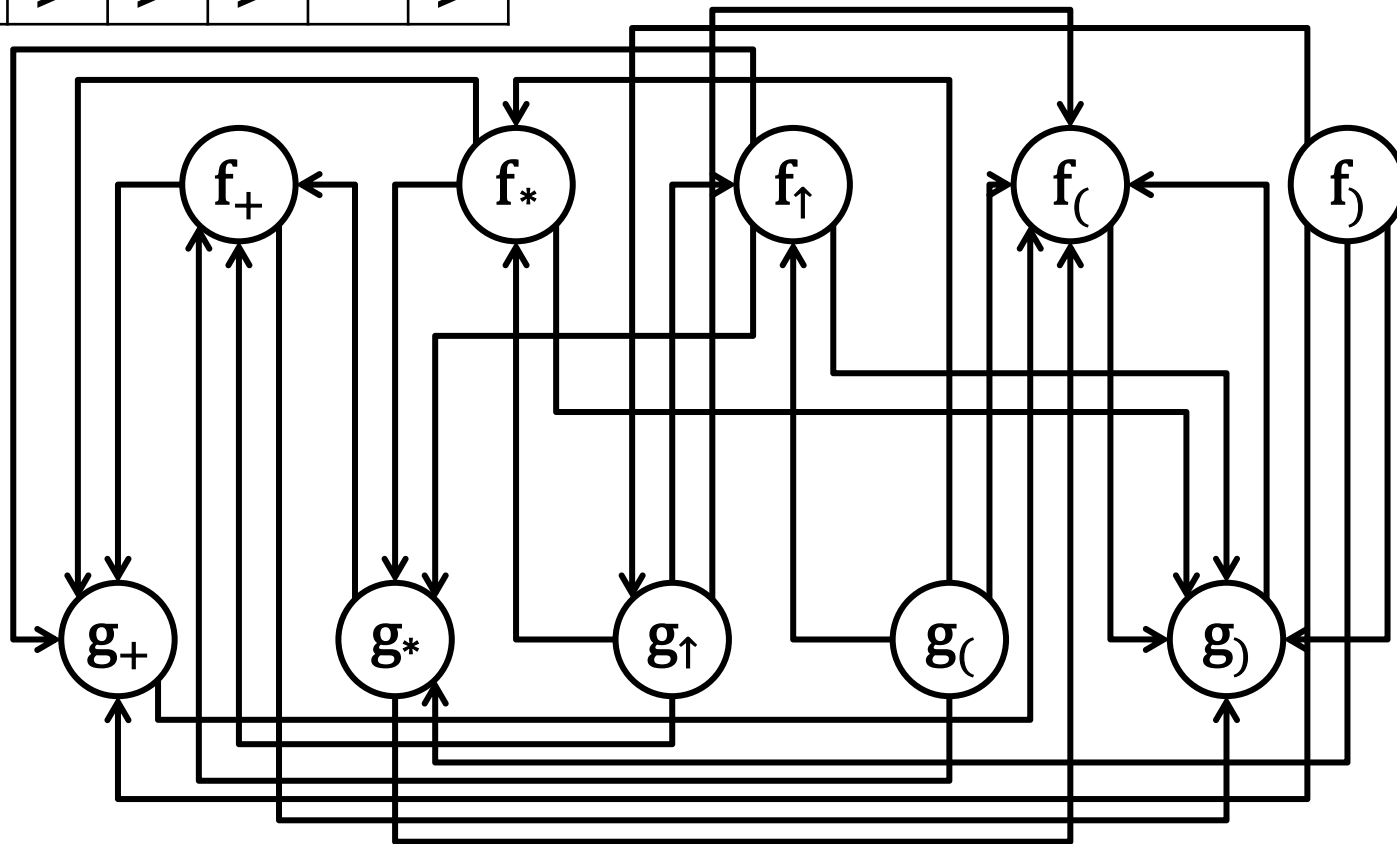
g f	+	*	↑	(	)
+	⋈	⋈	⋈	⋈	⋈
*	⋈	⋈	⋈	⋈	⋈
↑	⋈	⋈	⋈	⋈	⋈
(	⋈	⋈	⋈	⋈	⋈
)	⋈	⋈	⋈		⋈

	+	*	↑	(	)
f					
g					



$\begin{smallmatrix} g \\ f \end{smallmatrix} \diagdown$	$+$	$*$	$\uparrow$	$($	$)$
$+$	$\succ$	$\triangleleft$	$\triangleleft$	$\triangleleft$	$\succ$
$*$	$\succ$	$\succ$	$\triangleleft$	$\triangleleft$	$\succ$
$\uparrow$	$\succ$	$\succ$	$\triangleleft$	$\triangleleft$	$\succ$
$($	$\triangleleft$	$\triangleleft$	$\triangleleft$	$\triangleleft$	$\doteq$
$)$	$\succ$	$\succ$	$\succ$		$\succ$

	$+$	$*$	$\uparrow$	$($	$)$
f	4	6	6	2	9
g	3	5	8	8	2



5.3 LR分析法

- 自下而上分析

移进-归约法：用一个寄存符号的先进后出**栈**，把输入符号一个一个地**移进**栈里，当栈顶形成某个产生式的一个候选式时，即把栈顶的这一部分替换成（**归约**为）该产生式的左部符号(5.1.1)。

- LR分析法

- L: 从左到右扫描输入串;
- R: 最右推导的逆过程。

- Knuth, D. E. : Turing Award, 1974



October 25, 2005

5.3 LR分析法

- 5.3.1 LR分析器
- 5.3.2 LR(0)项目集族和LR(0)分析表的构造
- 5.3.3 SLR分析表的构造
- 5.3.4 规范LR分析表的构造
- 5.3.5 LALR分析表的构造

5.3.1 LR分析器

- LR分析器模型

- 栈:

栈的每一项内容包括状态 s 和文法符号 X 两部分;

$(s_0, \#)$: 分析开始前预先放到栈里的初始状态和句子括号;

s_m : 栈顶状态;

符号串 $X_1X_2...X_m$ 是至今已移进归约出的部分。

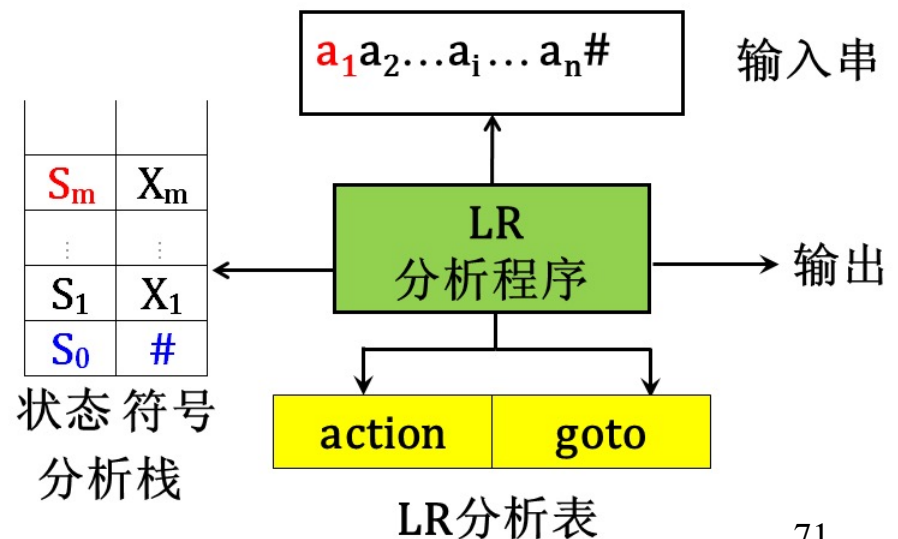


图5.4 LR分析器模型

5.3.1 LR分析器

- LR分析器模型

- LR分析表:

- **ACTION**[s , a]: 当状态 s 面临输入符号 a 时, 应采取什么动作.
 - **GOTO**[s , X]: 状态 s 面对文法符号 X 时, 下一状态是什么。GOTO[s , X] 定义了一个以文法符号为字母表的DFA。

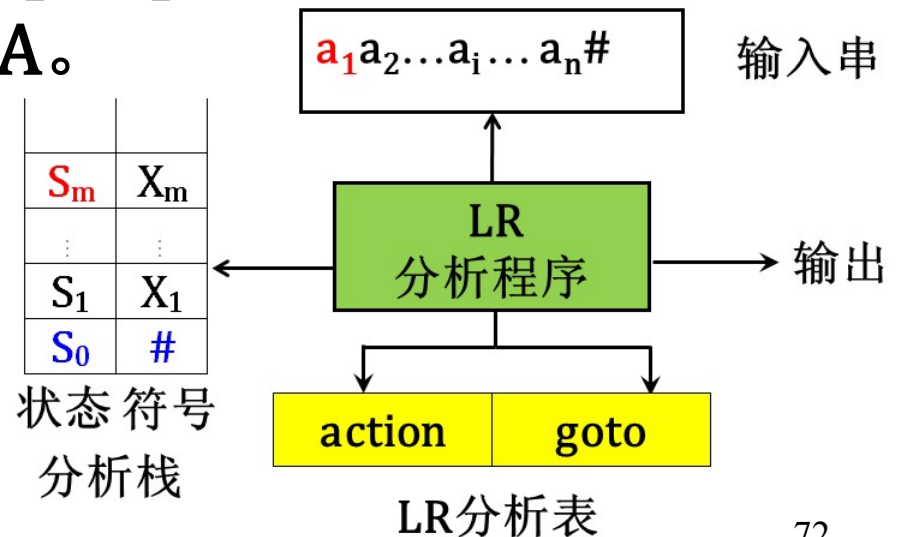


图5.4 LR分析器模型

状态	ACTION						GOTO		
	i	+	*	(	)	#	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

图5.5 LR分析表

- 每一项ACTION[s, a]所规定的四种动作:
 1. **移进** 把(s, a)的下一状态 s' 和输入符号 a 推进栈, 下一输入符号变成现行输入符号.
 2. **归约** 指用某产生式 $A \rightarrow \beta$ 进行归约. 假若 β 的长度为 r , 归约动作是, 去除栈顶 r 个项, 使状态 s_{m-r} 变成栈顶状态, 然后把(s_{m-r}, A)的下一状态 $s' = \text{GOTO}[s_{m-r}, A]$ 和文法符号 A 推进栈.
 3. **接受** 宣布分析成功, 停止分析器工作.
 4. **报错**

状态	ACTION						GOTO		
	i	+	*	(	)	#	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			

图5.5 LR分析表

5.3.1 LR分析器

- LR分析器模型
 - LR分析程序：

按栈顶状态 s 和现行输入符号 a 查询LR分析表，执行 $ACTION[s, a]$ 所规定的动作。

LR分析程序对所有的LR语法分析器都是一样的，不同的语法分析器LR分析表不同。

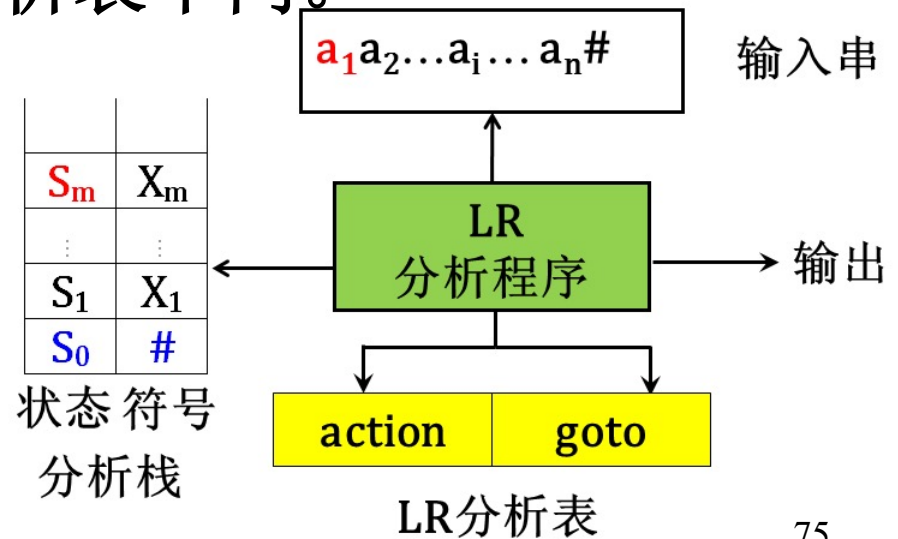


图5.4 LR分析器模型

5.3.1 LR分析器

文法G(E):

(1) $E \rightarrow E + T$

(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (E)$

(6) $F \rightarrow i$

状态	ACTION						GOTO		
	i	+	*	(	)	#	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

图5.5 LR分析表

文法G(E):

(1) $E \rightarrow E + T$

(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (E)$

(6) $F \rightarrow i$

步骤	状态	符号	输入串	动作
0	0	#	i*i+i#	s5
1	05	#i	*i+i#	r6
2	03	#F	*i+i#	r4
3	02	#T	*i+i#	s7
4	027	#T*	i+i#	s5
5	0275	#T*i	+i#	r6
6	02710	#T*F	+i#	r3
7	02	#T	+i#	r2
8	01	#E	+i#	s6
9	016	#E+	i#	s5
10	0165	#E+i	#	r6
11	0163	#E+F	#	r4
12	0169	#E+T	#	r1
13	01	#E	#	acc

状态	ACTION						GOTO		
	i	+	*	(	)	#	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

规范句型=符号栈中内容+剩余输入串。

图5.5 LR分析表

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- 字的前缀：字的任意首部。
字abc的前缀： ε 、a、ab、abc
- **活前缀**：规范句型的一个前缀，该前缀不含句柄之后的任何符号。
- 可归前缀：归约动作前符号栈中的内容称为可归前缀。

步骤	状态	符号	输入串	动作	活前缀
0	0	#	<u>i*i+i#</u>	s5	
1	05	# <u>i</u>	*i+i#	r6	ϵ, i
2	03	# <u>F</u>	*i+i#	r4	ϵ, F
3	02	# <u>T</u>	*i+i#	s7	
4	027	# <u>T*</u>	i+i#	s5	
5	0275	# <u>T*i</u>	+i#	r6	$\epsilon, T, T^*, T*i$
6	02710	# <u>T*F</u>	+i#	r3	$\epsilon, T, T^*, T*F$
7	02	# <u>T</u>	+i#	r2	ϵ, T
8	01	# <u>E</u>	+i#	s6	
9	016	# <u>E+</u>	i#	s5	
10	0165	# <u>E+i</u>	#	r6	$\epsilon, E, E+, E+i$
11	0163	# <u>E+F</u>	#	r4	$\epsilon, E, E+, E+F$
12	0169	# <u>E+T</u>	#	r1	$\epsilon, E, E+, E+T$
13	01	#E	#	acc	

规范句型=符号栈中内容+剩余输入串。

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- **活前缀**：规范句型的一个前缀，该前缀不含句柄之后的任何符号。

在LR分析工作过程中的任何时候，栈里的文法符号（自栈底而上） $X_1X_2...X_m$ 构成活前缀，把输入串的剩余部分配上之后即成为规范句型。

只要输入串的已扫描部分保持形成一个活前缀，那就意味着所扫描过的部分没有错。

首先，构造识别文法活前缀的有限自动机。

然后，将自动机转换为LR分析表。

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- 拓广文法(p107)

假定文法G是一个以S为开始符号的文法，构造一个G'，它包含了整个G，但它引进了一个不出现在G中的非终结符S'，并加进一个新产生式S'→S，而这个S'是G'的开始符号。

文法G(**E**):

$E \rightarrow aA | bB$

$A \rightarrow cA | d$

$B \rightarrow cB | d$

拓广文法5.7 G(**S'**):

$S' \rightarrow E$

$E \rightarrow aA | bB$

$A \rightarrow cA | d$

$B \rightarrow cB | d$

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- 拓广文法(p107)

目的：对某些右部含有开始符号的文法，在归约过程中能分清是否已归约到文法的最初开始符，还是在文法右部出现的开始符号，**拓广文法的开始符号 S' 只在左部出现**，确保了不会混淆。

文法G(**E**):

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow i$

拓广文法5.8 G(**S'**):

$S' \rightarrow E$

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow (E)$

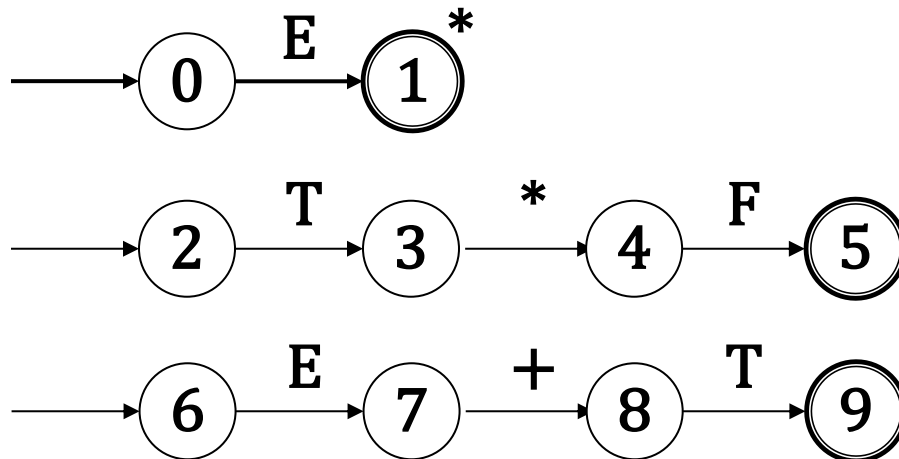
$F \rightarrow i$

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- 识别活前缀的有限自动机

如活前缀: E T^*F $E+T$

把终结符和非终结符都看成一个有限自动机的输入符号，每把一个符号进栈时看成已识别过该符号，而状态进行转换，当识别到可归前缀时，即在栈中形成句柄，则到达了识别句柄的终态。



5.3.2 LR(0)项目集族和LR(0)分析表的构造

- LR(0)项目

文法G的每个产生式的右部添加一个圆点称为G的一个LR(0)项目。

如: $A \rightarrow XYZ$ 有四个项目:

$A \rightarrow \cdot XYZ$

$A \rightarrow X \cdot YZ$

$A \rightarrow XY \cdot Z$

$A \rightarrow XYZ \cdot$

产生式 $A \rightarrow \varepsilon$ 只对应一个项目:

$A \rightarrow \cdot$

文法5.7 $G(S')$

$S' \rightarrow E$

$E \rightarrow aA | bB$

$A \rightarrow cA | d$

$B \rightarrow cB | d$

文法5.7的项目有：

- | | | |
|------------------------------|-------------------------------|------------------------------|
| 1. $S' \rightarrow \cdot E$ | 2. $S' \rightarrow E \cdot$ | |
| 3. $E \rightarrow \cdot aA$ | 4. $E \rightarrow a \cdot A$ | 5. $E \rightarrow aA \cdot$ |
| 6. $A \rightarrow \cdot cA$ | 7. $A \rightarrow c \cdot A$ | 8. $A \rightarrow cA \cdot$ |
| 9. $A \rightarrow \cdot d$ | 10. $A \rightarrow d \cdot$ | |
| 11. $E \rightarrow \cdot bB$ | 12. $E \rightarrow b \cdot B$ | 13. $E \rightarrow bB \cdot$ |
| 14. $B \rightarrow \cdot cB$ | 15. $B \rightarrow c \cdot B$ | 16. $B \rightarrow cB \cdot$ |
| 17. $B \rightarrow \cdot d$ | 18. $B \rightarrow d \cdot$ | |

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- LR(0)项目

一个项目指明了在分析过程的某时刻我们看到产生式多大一部分。

移进项目

待约项目

归约项目

接受项目

移进项目: $A \rightarrow \alpha \cdot a \beta$, $\alpha, \beta \in V^*$, $a \in V_T$

待约项目: $A \rightarrow \alpha \cdot B \beta$, $\alpha, \beta \in V^*$, $B \in V_N$

归约项目: $A \rightarrow \alpha \cdot$, $\alpha \in V^*$ 句柄已形成

接受项目: $S' \rightarrow \alpha \cdot$, $\alpha \in V^+$

文法
初态

文法5.7的项目有:

文法唯一的接受态

1. $S' \rightarrow \cdot E$
2. $S' \rightarrow E \cdot$
3. $E \rightarrow \cdot a A$
4. $E \rightarrow a \cdot A$
5. $E \rightarrow a A \cdot$
6. $A \rightarrow \cdot c A$
7. $A \rightarrow c \cdot A$
8. $A \rightarrow c A \cdot$
9. $A \rightarrow \cdot d$
10. $A \rightarrow d \cdot$
11. $E \rightarrow \cdot b B$
12. $E \rightarrow b \cdot B$
13. $E \rightarrow b B \cdot$
14. $B \rightarrow \cdot c B$
15. $B \rightarrow c \cdot B$
16. $B \rightarrow c B \cdot$
17. $B \rightarrow \cdot d$
18. $B \rightarrow d \cdot$

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- LR(0)项目集规范族

构成识别一个文法活前缀的DFA的项目集(状态)的全体称为文法的LR(0)项目集规范族。

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- LR(0)项目集规范族的构造
 - 项目集: CLOSURE函数
 - 状态转换: GO函数

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- LR(0)项目集规范族的构造

项目集：CLOSURE函数

假定 I 是文法 G' 的任一项目集，定义和构造 I 的闭包 $CLOSURE(I)$ 如下：

- I 的任何项目都属于 $CLOSURE(I)$ ；
- 若 $A \rightarrow \alpha \cdot B \beta$ 属于 $CLOSURE(I)$ ，那么，对任何关于 B 的产生式 $B \rightarrow \gamma$ ，项目 $B \rightarrow \cdot \gamma$ 也属于 $CLOSURE(I)$ ；
- 重复执行上述两步骤直至 $CLOSURE(I)$ 不再增大为止。

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- LR(0)项目集规范族的构造

项目集: CLOSURE函数

构造初态的闭包:

$I = \{S' \rightarrow \cdot E\}$

$CLOSURE(I) =$

$\{$
 $S' \rightarrow \cdot E,$
 $E \rightarrow \cdot aA,$
 $E \rightarrow \cdot bB$
 $\}$

文法5.7的项目有:

1. $S' \rightarrow \cdot E$	2. $S' \rightarrow E \cdot$	
3. $E \rightarrow \cdot aA$	4. $E \rightarrow a \cdot A$	5. $E \rightarrow aA \cdot$
6. $A \rightarrow \cdot cA$	7. $A \rightarrow c \cdot A$	8. $A \rightarrow cA \cdot$
9. $A \rightarrow \cdot d$	10. $A \rightarrow d \cdot$	
11. $E \rightarrow \cdot bB$	12. $E \rightarrow b \cdot B$	13. $E \rightarrow bB \cdot$
14. $B \rightarrow \cdot cB$	15. $B \rightarrow c \cdot B$	16. $B \rightarrow cB \cdot$
17. $B \rightarrow \cdot d$	18. $B \rightarrow d \cdot$	

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- LR(0)项目集规范族的构造

状态转换: GO函数

I是一个项目集, X是一个文法符号。

函数值GO(I, X)定义为:

$$GO(I, X) = CLOSURE(J)$$

其中

$J = \{\text{任何形如 } A \rightarrow \alpha \mathbf{X} \cdot \beta \text{ 的项目} \mid A \rightarrow \alpha \cdot \mathbf{X} \beta \text{ 属于 } I\}$ 。

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- LR(0)项目集规范族的构造

状态转换: GO函数

$CLOSURE(I) =$
 $\{$
 $S' \rightarrow \cdot E,$
 $E \rightarrow \cdot aA,$
 $E \rightarrow \cdot bB$
 $\}$
 $GO(I, a) = CLOSURE(J) =$
 $CLOSURE(\{E \rightarrow a \cdot A\})$
 $\{$
 $E \rightarrow a \cdot A,$
 $A \rightarrow \cdot cA,$
 $A \rightarrow \cdot d$
 $\}$

文法5.7的项目有:

- | | | |
|------------------------------|-------------------------------|------------------------------|
| 1. $S' \rightarrow \cdot E$ | 2. $S' \rightarrow E \cdot$ | |
| 3. $E \rightarrow \cdot aA$ | 4. $E \rightarrow a \cdot A$ | 5. $E \rightarrow aA \cdot$ |
| 6. $A \rightarrow \cdot cA$ | 7. $A \rightarrow c \cdot A$ | 8. $A \rightarrow cA \cdot$ |
| 9. $A \rightarrow \cdot d$ | 10. $A \rightarrow d \cdot$ | |
| 11. $E \rightarrow \cdot bB$ | 12. $E \rightarrow b \cdot B$ | 13. $E \rightarrow bB \cdot$ |
| 14. $B \rightarrow \cdot cB$ | 15. $B \rightarrow c \cdot B$ | 16. $B \rightarrow cB \cdot$ |
| 17. $B \rightarrow \cdot d$ | 18. $B \rightarrow d \cdot$ | |

- 文法 $G(S')$

$$S' \rightarrow E$$

$$E \rightarrow aA | bB$$

$$A \rightarrow cA | d$$

$$B \rightarrow cB | d$$

- $I_0 = \{S' \rightarrow \cdot E, E \rightarrow \cdot aA, E \rightarrow \cdot bB\}$

$$\begin{aligned} GO(I_0, E) &= \text{closure}(J) = \text{closure}(\{S' \rightarrow E \cdot\}) \\ &= \{S' \rightarrow E \cdot\} = I_1 \end{aligned}$$

$$\begin{aligned} GO(I_0, a) &= \text{closure}(J) = \text{closure}(\{E \rightarrow a \cdot A\}) \\ &= \{E \rightarrow a \cdot A, A \rightarrow \cdot cA, A \rightarrow \cdot d\} = I_2 \end{aligned}$$

$$\begin{aligned} GO(I_0, b) &= \text{closure}(J) = \text{closure}(\{E \rightarrow b \cdot B\}) \\ &= \{E \rightarrow b \cdot B, B \rightarrow \cdot cB, B \rightarrow \cdot d\} = I_3 \end{aligned}$$

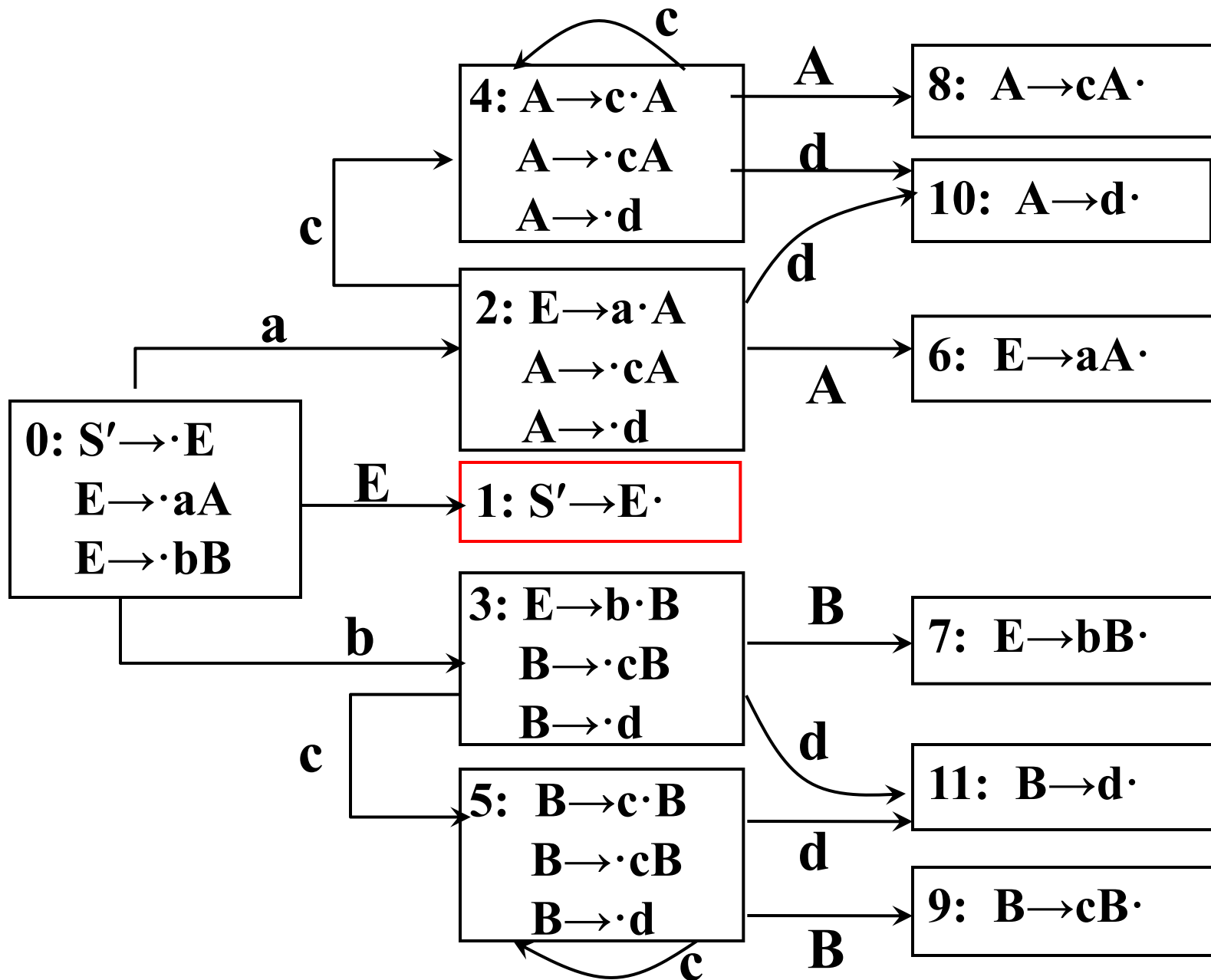


图5.7 识别活前缀的DFA

构造文法G的拓广文法G'的LR(0)项目集规范族算法:

PROCEDURE ITEMSETS(G');

BEGIN

 C:={CLOSURE({S'→·S})};

 REPEAT

 FOR C中每个项目集I和G'的每个符号X DO

 IF GO(I, X)非空且不属于C THEN

 把GO(I, X)放入C族中;

 UNTIL C 不再增大

END

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- LR(0)文法

一个文法G的拓广文法G'的活前缀识别自动机中的每个状态(项目集)**不存在**下述情况:

1) 既含移进项目又含归约项目,

如: $A \rightarrow \alpha \cdot a \beta$
 $B \rightarrow \gamma \cdot$

2) 含有多个归约项目。

如: $A \rightarrow \beta \cdot$
 $B \rightarrow \gamma \cdot$

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- LR(0)分析表的构造
 - 项目集规范族
 - 活前缀识别自动机的状态转换函数GO

令每个项目集 I_k 的下标 k 作为分析器的状态，包含项目 $S' \rightarrow \cdot E$ 的集合 I_k 的下标 k 为分析器的初态。

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- LR(0)分析表的构造

- (1) 若项目 $A \rightarrow \alpha \cdot a \beta$ 属于 I_k 且 $GO(I_k, a) = I_j$, a 为终结符, 则置 $ACTION[k, a] = sj$;
- (2) 若项目 $A \rightarrow \alpha \cdot$ 属于 I_k , 那么, 对 a 为任何终结符或 $\#$, 置 $ACTION[k, a] = rj$ (假定产生式 $A \rightarrow \alpha$ 是文法 G' 的第 j 个产生式);
- (3) 若项目 $S' \rightarrow E \cdot$ 属于 I_k , 置 $ACTION[k, \#] = acc$;
- (4) 若项目 $A \rightarrow \alpha \cdot A \beta$ 属于 I_k 且 $GO(I_k, A) = I_j$, A 为非终结符, 置 $GOTO[k, A] = j$;
- (5) 分析表中凡不能用规则1至4填入信息的空白格均置上“报错标志”。

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- LR(0)分析表的构造

文法5.7 $G(S')$

$S' \rightarrow E$

$E \rightarrow aA | bB$

$A \rightarrow cA | d$

$B \rightarrow cB | d$

文法5.7 $G(S')$

(0) $S' \rightarrow E$

(1) $E \rightarrow aA$

(2) $E \rightarrow bB$

(3) $A \rightarrow cA$

(4) $A \rightarrow d$

(5) $B \rightarrow cB$

(6) $B \rightarrow d$

• LR(0)分析表的构造

- (1) $A \rightarrow \alpha \cdot a \beta$ 属于 I_k 且 $GO(I_k, a) = I_j$, $ACTION[k, a] = sj$;
- (2) $A \rightarrow \alpha \cdot$ 属于 I_k , 对 a 为任何终结符或#, $ACTION[k, a] = rj$;
- (3) $S' \rightarrow E \cdot$ 属于 I_k , $ACTION[k, \#] = acc$;
- (4) $A \rightarrow \alpha \cdot A \beta$ 属于 I_k 且 $GO(I_k, A) = I_i$, $GOTO[k, A] = i$;
- (5) 空白格置上“报错标志”

文法5.7 $G(S')$

- (0) $S' \rightarrow E$
- (1) $E \rightarrow aA$
- (2) $E \rightarrow bB$
- (3) $A \rightarrow cA$
- (4) $A \rightarrow d$
- (5) $B \rightarrow cB$
- (6) $B \rightarrow d$

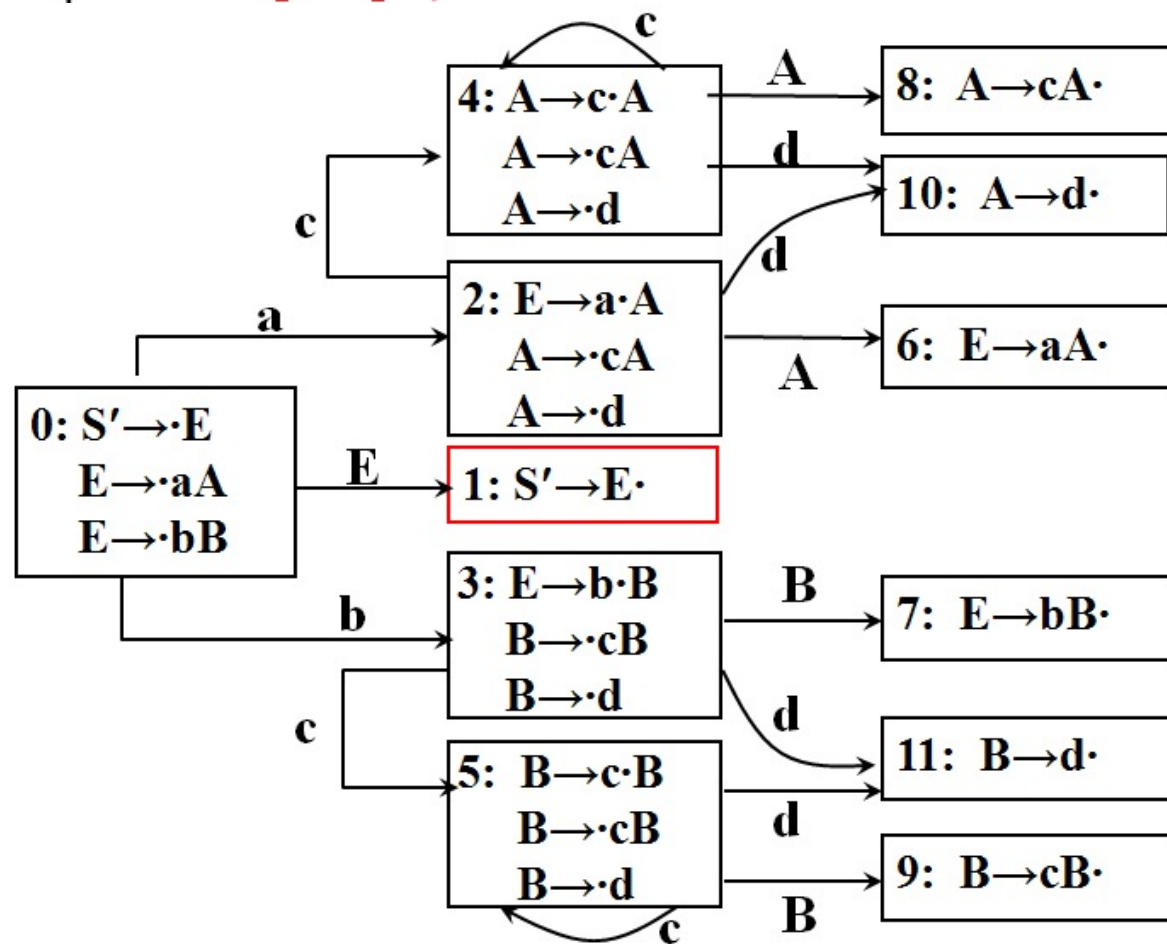


图5.7 识别活前缀的DFA

表5.4 LR(0)分析表

状态	ACTION					GOTO		
	a	b	c	d	#	E	A	B
0	s2	s3				1		
1					acc			
2			s4	s10			6	
3			s5	s11				7
4			s4	s10			8	
5			s5	s11				9
6	r1	r1	r1	r1	r1			
7	r2	r2	r2	r2	r2			
8	r3	r3	r3	r3	r3			
9	r5	r5	r5	r5	r5			
10	r4	r4	r4	r4	r4			
11	r6	r6	r6	r6	r6			

文法5.7 $G(S')$

(0) $S' \rightarrow E$

(1) $E \rightarrow aA$

(2) $E \rightarrow bB$

(3) $A \rightarrow cA$

(4) $A \rightarrow d$

(5) $B \rightarrow cB$

(6) $B \rightarrow d$

5.3.2 LR(0)项目集族和LR(0)分析表的构造

- LR(0)分析器的工作过程

输入串 bccd# 的LR(0) 分析过程

步骤	状态	符号	输入串	动作
0	0	#	bccd#	
1	03	#b	ccd#	s3
2	035	#bc	cd#	s5
3	0355	#bcc	d#	s5
4	0355 <u>11</u>	#bccd	#	s11
5	03559	#bccB	#	r6: B→d
6	0359	#bcB	#	r5: B→cB
7	037	#bB	#	r5: B→cB
8	01	#E	#	r2: E→bB acc

5.3.3 SLR分析表的构造

- LR(0)文法简单，文法的活前缀识别自动机的每个状态(项目集)都不含冲突性的项目。

5.3.3 SLR分析表的构造

- 例5.11

文法 $G(E)$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid i$

拓广文法5.8 $G(S')$

(0) $S' \rightarrow E$

(1) $E \rightarrow E + T$

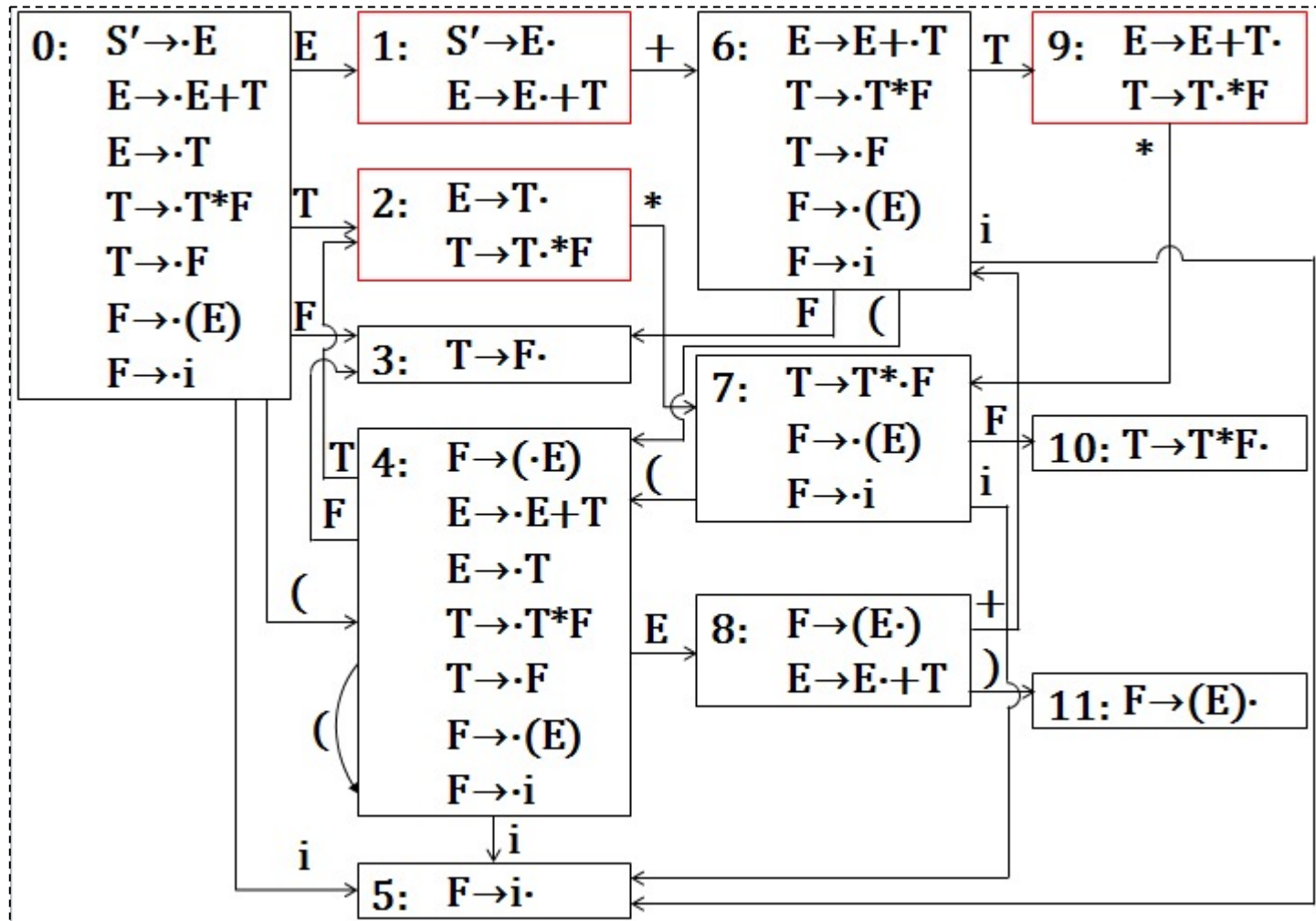
(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (E)$

(6) $F \rightarrow i$



5.3.3 SLR分析表的构造

- I_1 、 I_2 和 I_9 都含有“移进—归约”冲突，文法5.8不是LR(0)的。

1: $S' \rightarrow E \cdot$
 $E \rightarrow E \cdot + T$

2: $E \rightarrow T \cdot$
 $T \rightarrow T \cdot * F$

9: $E \rightarrow E + T \cdot$
 $T \rightarrow T \cdot * F$

状态	ACTION						GOTO		
	i	+	*	(	)	#	E	T	F
0									
1		s6				acc			
2	r2	r2	s7,r2	r2	r2	r2			
9	r1	r1	s7,r1	r1	r1	r1			

图5.5 LR分析表

5.3.3 SLR分析表的构造

- 许多冲突性的动作都可能通过考察**非终结符的 FOLLOW集**而获解决。

2: $E \rightarrow T \cdot$
 $T \rightarrow T \cdot * F$

$\text{FOLLOW}(E) = \{ \#,), + \}$

状态	ACTION						GOTO		
	i	+	*	(	)	#	E	T	F
0									
1		s6				acc			
2		r2	s7		r2	r2			
9	r1	r1	s7,r1	r1	r1	r1			

图5.5 LR分析表

5.3.3 SLR分析表的构造

- 许多冲突性的动作都可能通过考察**非终结符的 FOLLOW集**而获解决。

9: $E \rightarrow E + T \cdot$
 $T \rightarrow T * F$

$\text{FOLLOW}(E) = \{ \#,), + \}$

状态	ACTION						GOTO		
	i	+	*	(	)	#	E	T	F
0									
1		s6				acc			
2		r2	s7		r2	r2			
9		r1	s7		r1	r1			

图5.5 LR分析表

5.3.3 SLR分析表的构造

假定一个LR(0)规范族中含有如下的一个项目集(状态):

$$I = \{ \begin{array}{l} X \rightarrow \alpha \cdot b \beta, \\ A \rightarrow \gamma \cdot, \\ B \rightarrow \delta \cdot \end{array} \}$$

FOLLOW(A)和FOLLOW(B)的交集为 $\emptyset$ ，且不含b，那么，当状态I面临任何输入符号 a 时，可以:

1. 若 $a=b$ ，则移进;
2. 若 $a \in \text{FOLLOW}(A)$ ，用产生式 $A \rightarrow \gamma$ 进行归约;
3. 若 $a \in \text{FOLLOW}(B)$ ，用产生式 $B \rightarrow \delta$ 进行归约;
4. 此外，报错。

5.3.3 SLR分析表的构造

- SLR(1)解决办法

假定LR(0)规范族的一个项目集含有 m 个移进项目和 n 个归约项目：

$I = \{A_1 \rightarrow \alpha_1 \cdot a_1 \beta_1, A_2 \rightarrow \alpha_2 \cdot a_2 \beta_2, \dots, A_m \rightarrow \alpha_m \cdot a_m \beta_m, B_1 \rightarrow \gamma_1 \cdot, B_2 \rightarrow \gamma_2 \cdot, \dots, B_n \rightarrow \gamma_n \cdot\}$ 。

如果集合 $\{a_1, \dots, a_m\}$, $\text{FOLLOW}(B_1)$, ..., $\text{FOLLOW}(B_n)$ 两两不相交(包括不得有两个 FOLLOW 集合有 $\#$)，则：

1. 若 a 是某个 a_i , $i=1, 2, \dots, m$, 则移进;
2. 若 $a \in \text{FOLLOW}(B_i)$, $i=1, 2, \dots, n$, 则用产生式 $B_i \rightarrow \gamma_i$ 进行归约;
3. 此外，报错。

5.3.3 SLR分析表的构造

- 构造SLR(1)分析表

首先把G拓广为G'，对G'构造LR(0)项目集规范族C和活前缀识别自动机的状态转换函数GO.

然后使用C和GO，按以下算法构造SLR(1)分析表：

假定 $C = \{ I_0, I_1, \dots, I_n \}$ ，令每个项目集 I_k 的下标 k 作为分析器的状态，包含项目 $S' \rightarrow \cdot E$ 的集合 I_k 的下标 k 为分析器的初态。

5.3.3 SLR分析表的构造

- 构造SLR(1)分析表

函数ACTION和GOTO按如下方法构造：

1. 若项目 $A \rightarrow \alpha \cdot a \beta$ 属于 I_k 且 $GO(I_k, a) = I_j$, a 为终结符, 则置 $ACTION[k, a] = sj$;

2. 若项目 $A \rightarrow \alpha \cdot$ 属于 I_k , 那么, 对 a 为任何终结符或#, $a \in FOLLOW(A)$, 置 $ACTION[k, a] = rj$; 其中, 假定 $A \rightarrow \alpha$ 为文法 G' 的第 j 个产生式;

3. 若项目 $S' \rightarrow E \cdot$ 属于 I_k , 则置 $ACTION[k, \#] = acc$;

4. 若项目 $A \rightarrow \alpha \cdot A \beta$ 属于 I_k 且 $GO(I_k, A) = I_j$, A 为非终结符, 则置 $GOTO[k, A] = j$;

5. 分析表中凡不能用规则1至4填入信息的空白格均置上“出错标志”。

• 构造SLR(1)分析表

函数ACTION和GOTO按如下方法构造:

- 1. $A \rightarrow \alpha \cdot a \beta$ 属于 I_k 且 $GO(I_k, a) = I_j$, $ACTION[k, a] = sj$;
- 2. $A \rightarrow \alpha \cdot$ 属于 I_k , a 或 $\# \in FOLLOW(A)$, $ACTION[k, a] = rj$;
- 3. $S' \rightarrow E \cdot$ 属于 I_k , $ACTION[k, \#] = acc$;
- 4. $A \rightarrow \alpha \cdot A \beta$ 属于 I_k 且 $GO(I_k, A) = I_j$, $GOTO[k, A] = j$;

FOLLOW	
S'	{ # }
E	{ #, +,) }
T	{ #, +,), * }
F	{ #, +,), * }

拓广文法5.8 $G(S')$

(0) $S' \rightarrow E$

(1) $E \rightarrow E + T$

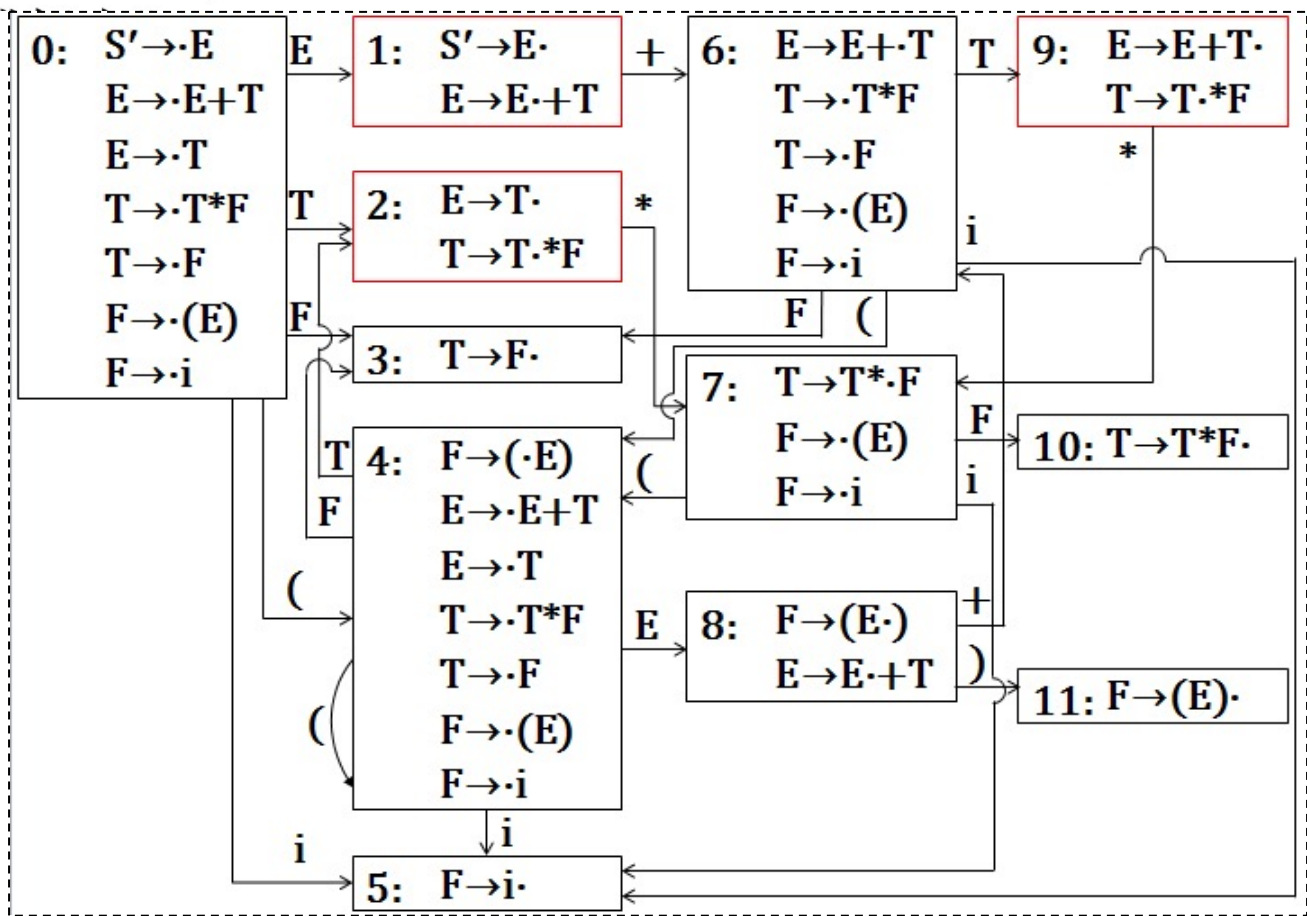
(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (E)$

(6) $F \rightarrow i$



状态	ACTION						GOTO		
	i	+	*	(	)	#	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

图5.5 SLR(1)分析表

5.3.3 SLR分析表的构造

- 按上述方法构造出的ACTION与GOTO表如果不含多重入口，则称该文法为SLR(1)文法。
- 使用SLR表的分析器叫做一个SLR分析器。
- 每个SLR(1)文法都是无二义的。但也存在许多无二义文法不是SLR(1)的。

5.3.3 SLR分析表的构造

- 文法5.9 $G(S')$:

(0) $S' \rightarrow S$

(1) $S \rightarrow L = R$

(2) $S \rightarrow R$

(3) $L \rightarrow *R$

(4) $L \rightarrow i$

(5) $R \rightarrow L$

这个文法的LR(0)项目集规范族为:

$I_0: S' \rightarrow \cdot S$
 $S \rightarrow \cdot L = R$
 $S \rightarrow \cdot R$
 $S \rightarrow \cdot * R$
 $L \rightarrow \cdot i$
 $R \rightarrow \cdot L$

$I_1: S' \rightarrow S \cdot$

$I_2: S \rightarrow L \cdot = R$
 $R \rightarrow L \cdot$

$I_3: S \rightarrow R \cdot$

$I_4: L \rightarrow * \cdot R$
 $R \rightarrow \cdot L$
 $L \rightarrow \cdot * R$
 $L \rightarrow \cdot i$

$I_5: L \rightarrow i \cdot$

$I_6: S \rightarrow L = \cdot R$
 $R \rightarrow \cdot L$
 $L \rightarrow \cdot * R$
 $L \rightarrow \cdot i$

$I_7: L \rightarrow * R \cdot$

$I_8: R \rightarrow L \cdot$

$I_9: S \rightarrow L = R \cdot$

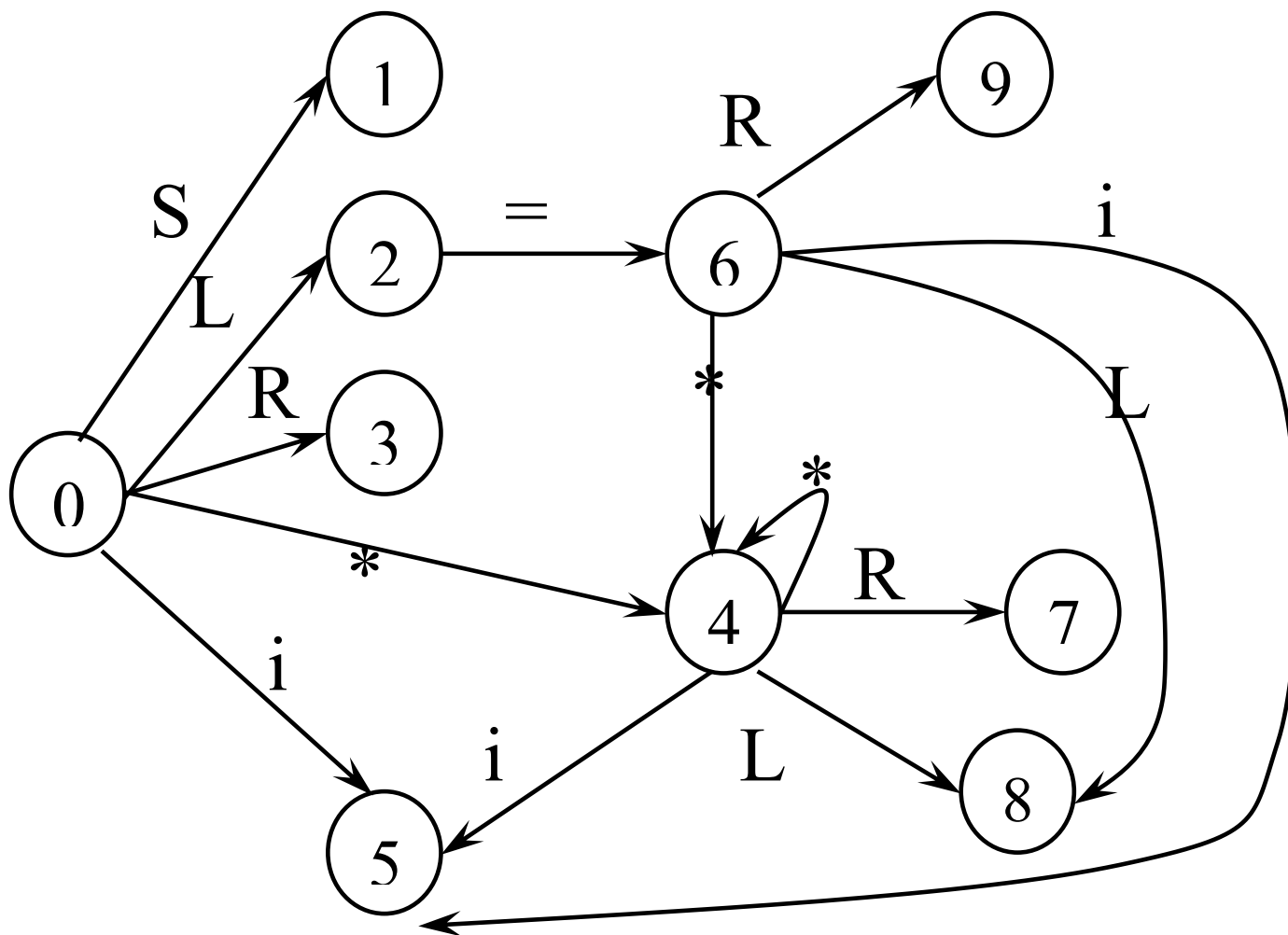


图5.9 文法(5.9)活前缀识别器

- I_2 含有“移进—归约”冲突

$$\text{FOLLOW}(R) = \{\#, =\}$$

$$I_2: \begin{array}{l} S \rightarrow L \cdot = R \\ R \rightarrow L \cdot \end{array}$$

- 产生冲突的原因：SLR分析法未包含足够的“展望”信息。
- 解决方法：让每个状态含有更多的“展望”信息，对状态进行分裂，使得LR分析器的每个状态对于归约项目 $A \rightarrow \alpha$ 能够确切地指出，当 α 后跟哪些终结符时才容许把 α 归约为 A 。

5.3.4 规范LR分析表的构造

- LR(k)项目:

项目的一般形式是 $[A \rightarrow \alpha \cdot \beta, a_1 a_2 \dots a_k]$,

$A \rightarrow \alpha \cdot \beta$: LR(0)项目

每个a: 终结符

$a_1 a_2 \dots a_k$: 向前搜索字符串(或展望串)。

向前搜索字符串仅对归约项目 $[A \rightarrow \alpha \cdot, a_1 a_2 \dots a_k]$ 有意义。

对于任何移进或待约项目 $[A \rightarrow \alpha \cdot \beta, a_1 a_2 \dots a_k]$, $\beta \neq \epsilon$, 搜索字符串 $a_1 a_2 \dots a_k$ 没有作用。

5.3.4 规范LR分析表的构造

- 归约项目 $[A \rightarrow \alpha \cdot, a_1 a_2 \dots a_k]$ 意味着：当它所属的状态呈现在栈顶且后续的 k 个输入符号为 $a_1 a_2 \dots a_k$ 时，才可以把栈顶上的 α 归约为 A 。
- 我们只对 $k \leq 1$ 的情形感兴趣，对多数程序语言的语法来说，向前搜索(展望)一个符号就多半可以确定“移进”或“归约”。

即：LR(1)项目：

$[A \rightarrow \alpha \cdot, a]$

5.3.4 规范LR分析表的构造

- 构造LR(1)项目集族
 - 闭包函数CLOSURE
 - 状态转换函数GO

• 项目集I 的闭包CLOSURE(I)构造方法:

1. I的任何项目都属于CLOSURE(I)。

2. 若项目 $[A \rightarrow \alpha \cdot B \beta, a]$ 属于CLOSURE(I), $B \rightarrow \gamma$ 是一个产生式, 那么, 对于 $\text{FIRST}(\beta a)$ 中的每个终结符 b , 如果 $[B \rightarrow \cdot \gamma, b]$ 原来不在CLOSURE(I)中, 则把它加进去。

3. 重复执行步骤2, 直至CLOSURE(I)不再增大为止。

5.3.4 规范LR分析表的构造

- 构造LR(1)项目集族

项目集: CLOSURE函数

$$I_0 = \text{CLOSURE}(\{[S' \rightarrow \cdot S, \#]\})$$
$$= \{ \begin{array}{l} [S' \rightarrow \cdot S, \#], \\ [S \rightarrow \cdot BB, \#], \\ [B \rightarrow \cdot aB, a/b], \\ [B \rightarrow \cdot b, a/b] \end{array} \}$$

(5.10)的拓广文法 $G(S')$:

- (0) $S' \rightarrow S$
- (1) $S \rightarrow BB$
- (2) $B \rightarrow aB$
- (3) $B \rightarrow b$

$$\text{FIRST}(S') = \{ a, b \}$$
$$\text{FIRST}(S) = \{ a, b \}$$
$$\text{FIRST}(B) = \{ a, b \}$$

5.3.4 规范LR分析表的构造

- 构造LR(1)项目集族

状态转换: GO函数

令I是一个项目集, X是一个文法符号, 函数GO(I, X)定义为:

$$GO(I, X) = CLOSURE(J)$$

其中

$$J = \{ \text{任何形如 } [A \rightarrow \alpha X \cdot \beta, a] \text{ 的项目} \\ | [A \rightarrow \alpha \cdot X \beta, a] \in I \}$$

5.3.4 规范LR分析表的构造

- 构造LR(1)项目集族

状态转换: GO函数

$$I_0 = \text{CLOSURE}(\{[S' \rightarrow \cdot S, \#]\})$$
$$= \{ \begin{array}{l} [S' \rightarrow \cdot S, \#], \\ [S \rightarrow \cdot BB, \#], \\ [B \rightarrow \cdot aB, a/b], \\ [B \rightarrow \cdot b, a/b] \end{array} \}$$

(5.10)的拓广文法 $G(S')$:

- (0) $S' \rightarrow S$
- (1) $S \rightarrow BB$
- (2) $B \rightarrow aB$
- (3) $B \rightarrow b$

$$I_1 = \text{GO}(I_0, S) = \text{CLOSURE}(\{[S' \rightarrow S \cdot, \#]\}) = \{[S' \rightarrow S \cdot, \#]\}$$
$$I_2 = \text{GO}(I_0, B) = \text{CLOSURE}(\{[S \rightarrow B \cdot B, \#]\})$$
$$= \{[S \rightarrow B \cdot B, \#], [B \rightarrow \cdot aB, \#], [B \rightarrow \cdot b, \#]\}$$

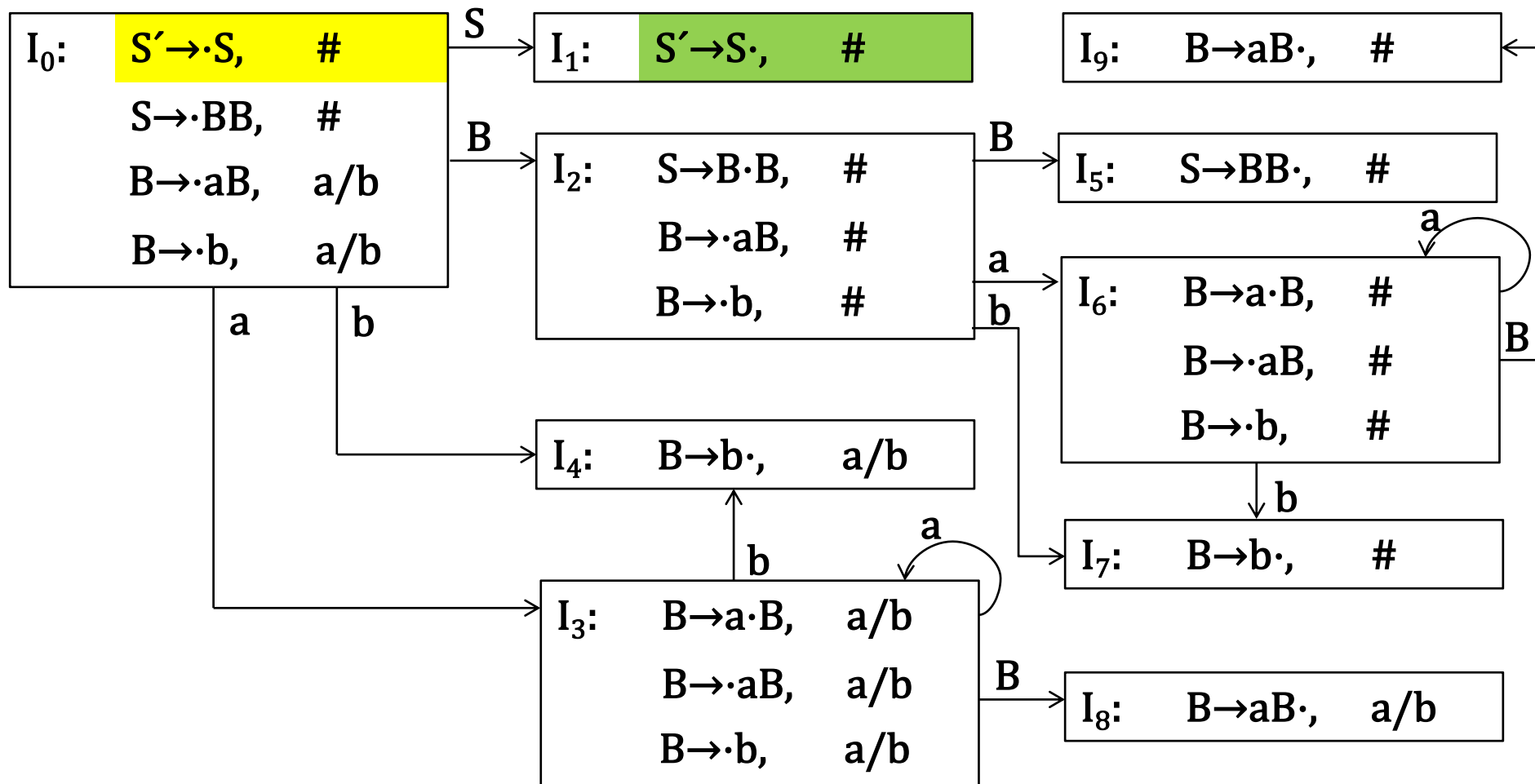


图5.10 文法(5.10)LR(1)项目集和GO函数

- 构造LR(1)分析表的算法。
 - 令每个 I_k 的下标 k 为分析表的状态，令含有 $[S' \rightarrow \cdot S, \#]$ 的 I_k 的 k 为分析器的初态。

- 动作ACTION和状态转换GOTO构造如下：
 1. 若项目 $[A \rightarrow \alpha \cdot a \beta, b]$ 属于 I_k 且 $GO(I_k, a) = I_j$, a 为终结符, 则置 $ACTION[k, a] = sj$ 。
 2. 若项目 $[A \rightarrow \alpha \cdot, a]$ 属于 I_k , 则置 $ACTION[k, a] = rj$; 其中假定 $A \rightarrow \alpha$ 为文法 G' 的第 j 个产生式。
 3. 若项目 $[S' \rightarrow S \cdot, \#]$ 属于 I_k , 则置 $ACTION[k, \#] = acc$ 。
 4. 若项目 $[A \rightarrow \alpha \cdot A \beta, b]$ 属于 I_k 且 $GO(I_k, A) = I_j$, 则置 $GOTO[k, A] = j$ 。
 5. 分析表中凡不能用规则1至4填入信息的空白栏均填上“出错标志”。

动作ACTION和状态转换GOTO构造如下：

1. $[A \rightarrow \alpha \cdot a \beta, b]$ 属于 I_k 且 $GO(I_k, a) = I_j$, **ACTION** $[k, a] = sj$ 。
2. $[A \rightarrow \alpha \cdot, a]$ 属于 I_k , **ACTION** $[k, a] = rj$ 。
3. $[S' \rightarrow S \cdot, \#]$ 属于 I_k , **ACTION** $[k, \#] = acc$ 。
4. $[A \rightarrow \alpha \cdot A \beta, b]$ 属于 I_k 且 $GO(I_k, A) = I_j$, **GOTO** $[k, A] = j$ 。
5. 空白栏均填上“出错标志”。

(5.10) $G(S')$:

(0) $S' \rightarrow S$

(1) $S \rightarrow BB$

(2) $B \rightarrow aB$

(3) $B \rightarrow b$

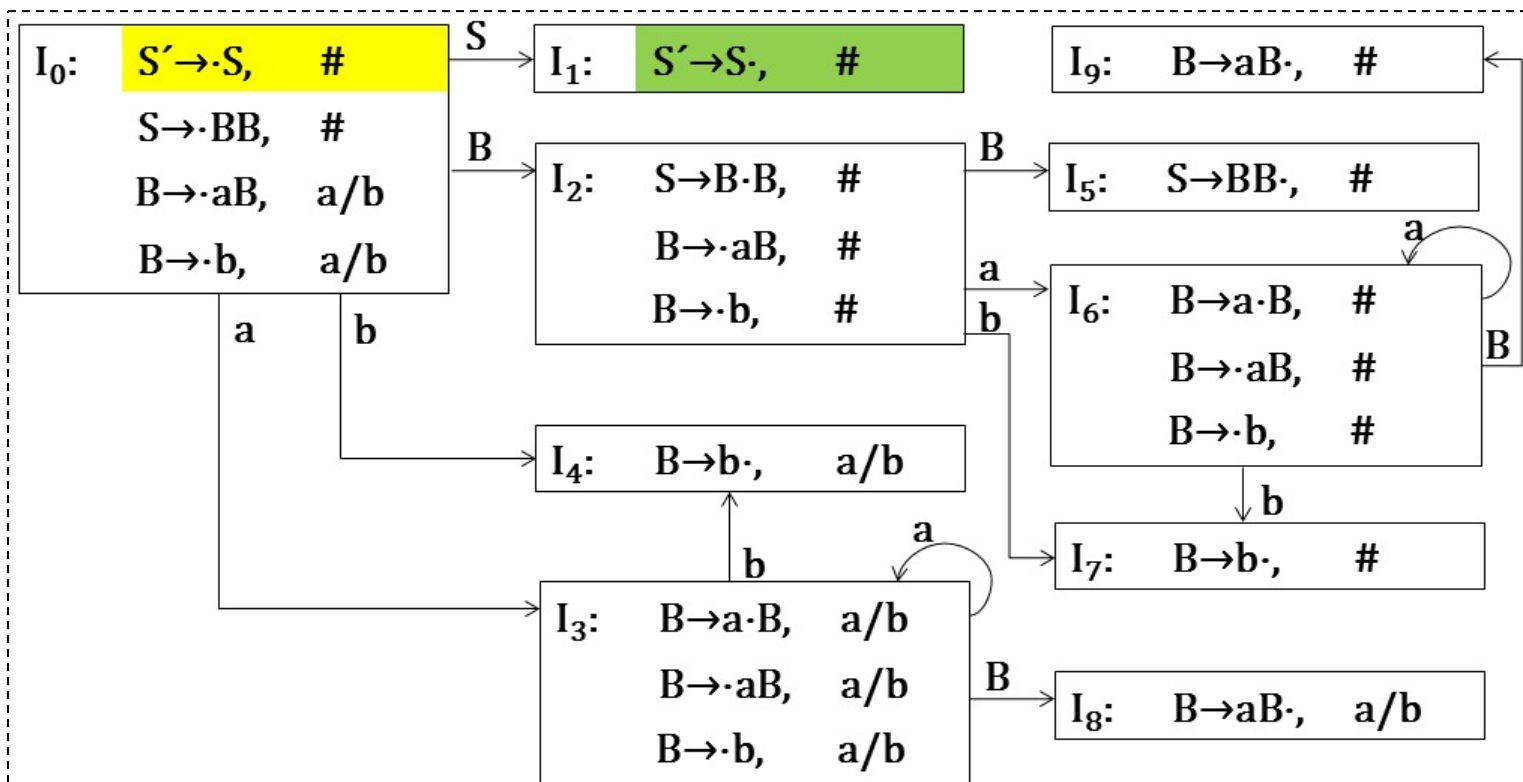


图5.10 文法5.10的LR(1)项目集和GOTO函数

LR(1)分析表为:

状态	ACTION			GOTO	
	<i>a</i>	<i>b</i>	#	<i>S</i>	<i>B</i>
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

- 按上述算法构造的分析表，若不存在多重定义的入口(即，动作冲突)的情形，则称它是文法G的一张**规范的LR(1)分析表**。
- 使用这种分析表的分析器叫做一个**规范的LR分析器**。
- 具有规范的LR(1)分析表的文法称为一个**LR(1)文法**。
- LR(1)状态比SLR多

5.3.5 LALR分析表的构造

心：一个LR(1)项目可以看成两个部分组成，一部分和LR(0)项目相同，这部分称它为心，另一部分为向前搜索符。

同心集：对于两个LR(1)项目集，如果除去搜索符之后，这两个集合是相同的，则称这两个集合是同心集。

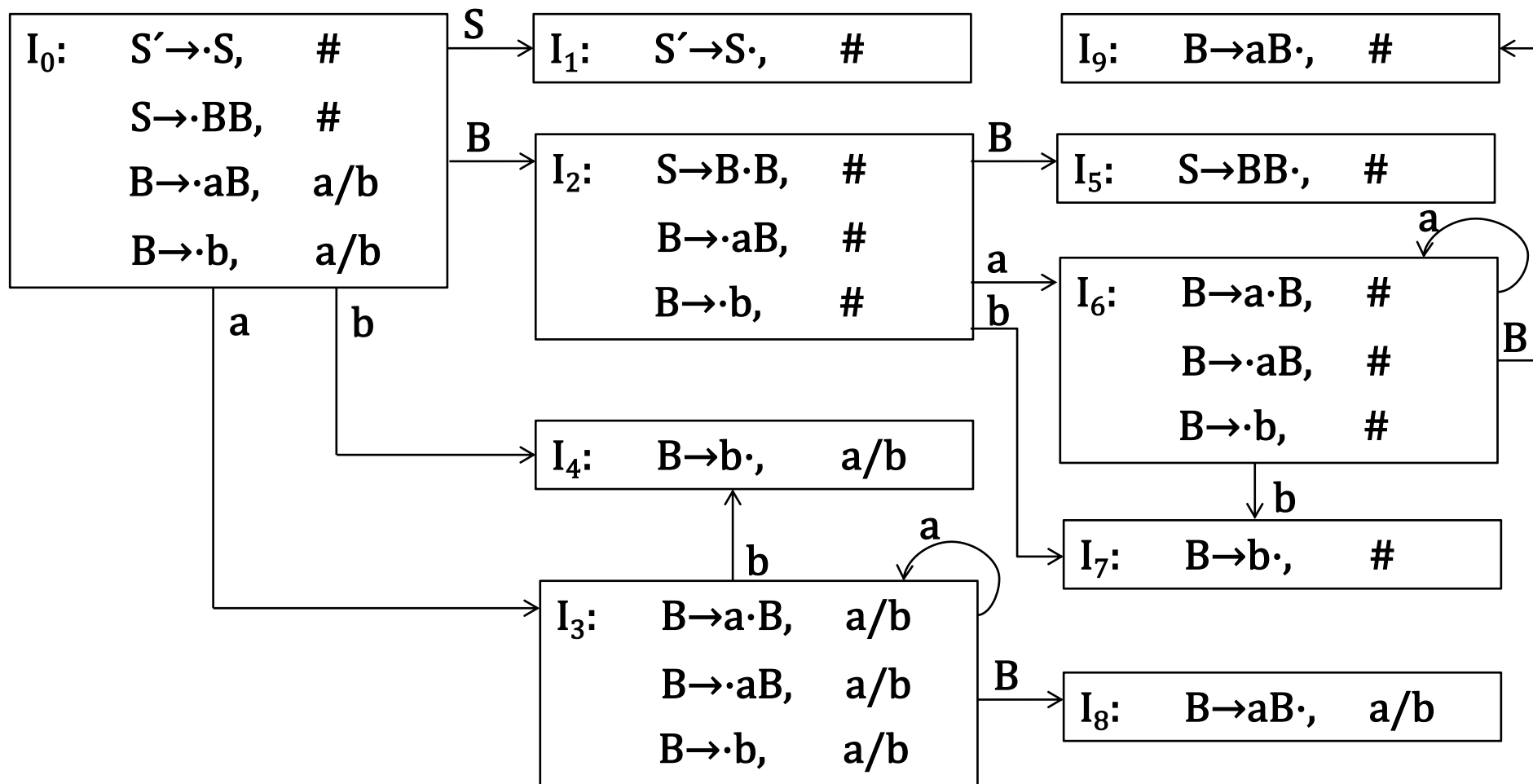


图5.10 文法(5.10)LR(1)项目集和GO函数

5.3.5 LALR分析表的构造

LALR分析表的构造:

1. 构造文法G的LR(1)项目集族 $C = \{ I_0, I_1, \dots, I_n \}$;
2. 把所有同心集合并在一起, 记 $C' = \{ J_0, J_1, \dots, J_n \}$ 为合并后的新族。含有项目 $[S' \rightarrow \cdot S, \#]$ 的 J_k 为分析表的初态。

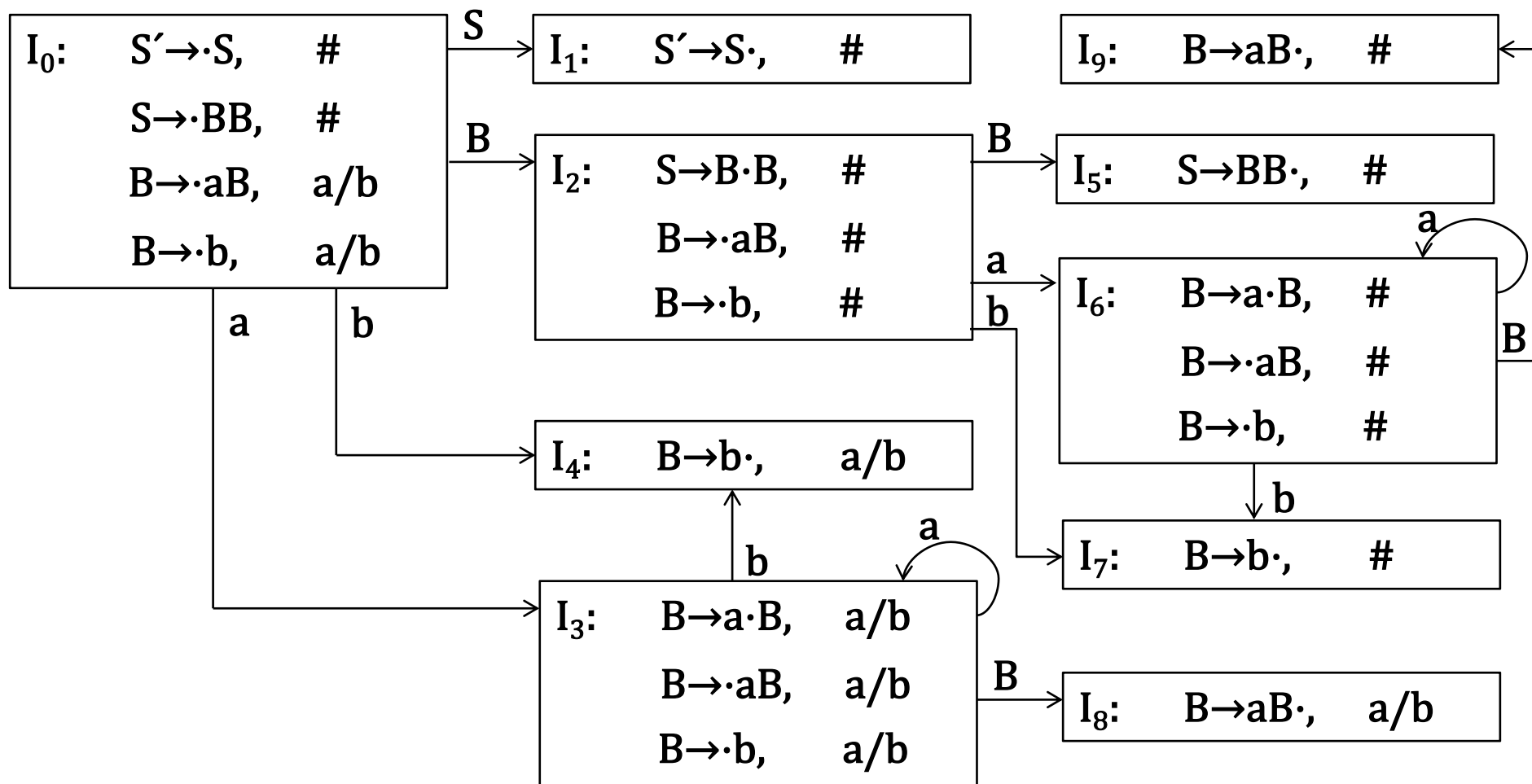


图5.10 文法(5.10)LR(1)项目集和GO函数

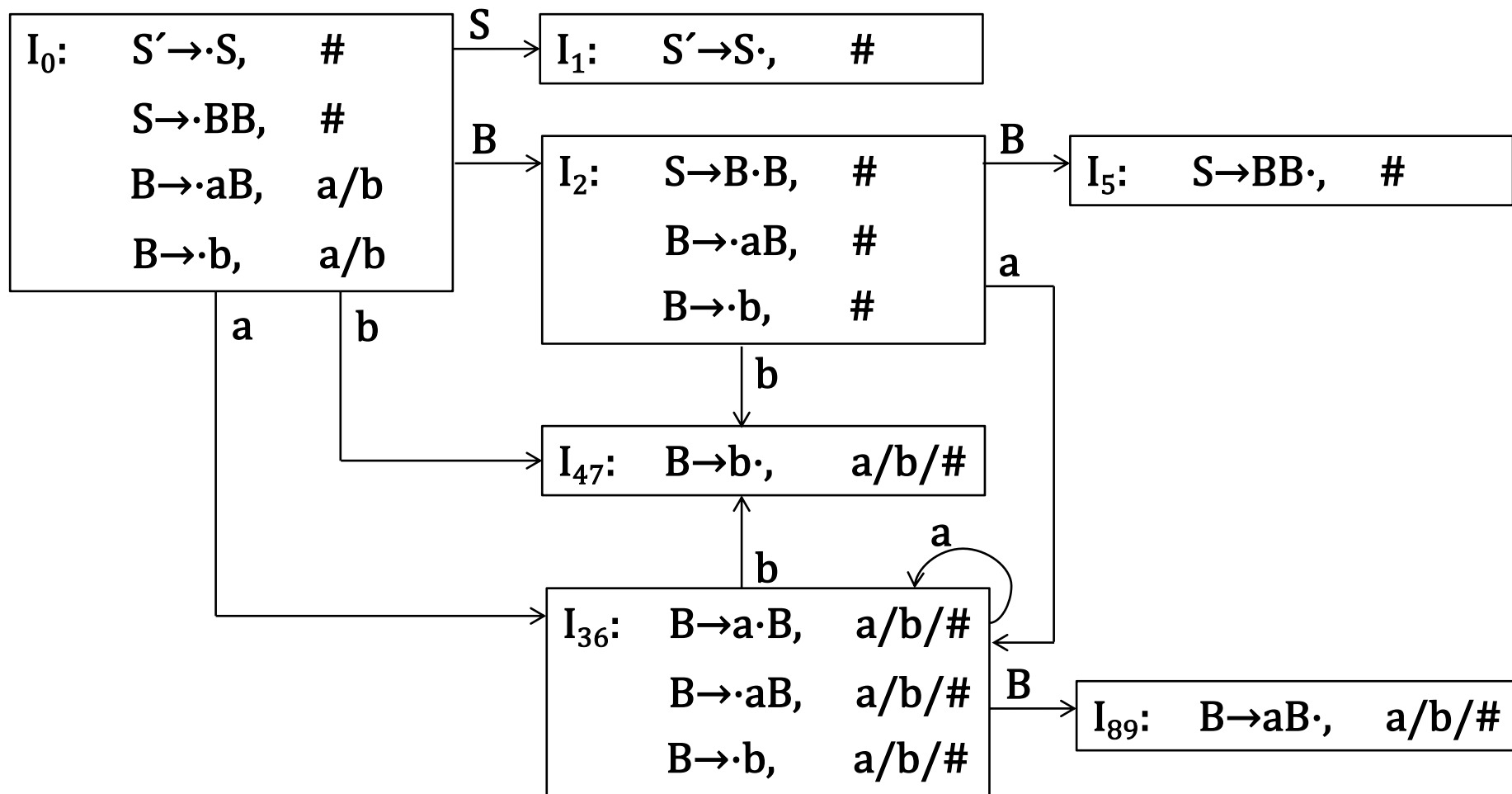


图5.10 文法(5.10)LALR(1)项目集和GO函数

5.3.5 LALR分析表的构造

LALR分析表的构造:

3. 从 C' 构造 ACTION 表:

(1) 若项目 $[A \rightarrow \alpha \cdot a \beta, b] \in J_k$ 且 $GO(J_k, a) = J_j$, a 为终结符, 则置 $ACTION[k, a] = sj$ 。

(2) 若项目 $[A \rightarrow \alpha \cdot, a] \in J_k$, 则置 $ACTION[k, a] = rj$; 其中假定 $A \rightarrow \alpha$ 为文法 G' 的第 j 个产生式。

(3) 若项目 $[S' \rightarrow S \cdot, \#] \in J_k$, 则置 $ACTION[k, \#] = acc$ 。

4. GOTO 表的构造: 若 $GO(J_k, A) = J_i$, 则置 $GOTO[k, A] = i$ 。

5. 分析表中凡不能用3、4填入信息的空白格均填上“出错标志”。

表5.6 文法(5.10)LALR(1)分析表

状态	ACTION			GOTO	
	a	b	#	S	B
0	s36	s47		1	2
1			acc		
2	s36	s47			5
36	s36	s47			89
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

5.3.5 LALR分析表的构造

- 经上述步骤构造的分析表若不存在冲突，则称它为文法的LALR分析表，存在这种分析表的文法称为一个LALR(1)文法。

5.3.7 LR分析中的出错处理

- 发现语法错误：输入符号既不能移入栈顶，栈内元素又不能归约。
- 处理方法：
 - 插入、删除或修改
如： `a[1,2 := 3.14;`
插入一个`]`, `a[1,2 ]:= 3.14;`
 - 将含有错误的短语局部化

5.3.7 LR分析中的出错处理

文法(5.11) $G(E)$:

(1)

$E \rightarrow E + E$

(2) $E \rightarrow E * E$

(3) $E \rightarrow (E)$

(4) $E \rightarrow i$

表5.9 文法5.11的LR(1)分析表
(含出错处理子程序)

状态	ACTION						GOTO
	i	+	*	(	)	#	E
0	s3	e1	e1	s2	e2	e1	1
1	e3	s4	s5	e3	e2	acc	
2	s3	e1	e1	s2	e2	e1	6
3	r4	r4	r4	r4	r4	r4	
4	s3	e1	e1	s2	e2	e1	7
5	s3	e1	e1	s2	e2	e1	8
6	e3	s4	s5	e3	s9	e4	
7	r1	r1	s5	r1	r1	r1	
8	r2	r2	r2	r2	r2	r2	
9	r3	r3	r3	r3	r3	r3	

5.3.7 LR分析中的出错处理

表5.9 文法5.11的LR(1)分析表
(含出错处理子程序)

e1: /* 处在状态0,2,4,5时,
要求输入符号为运算
量的首符, 如i或左括
号。
当遇到+、*或#等,
调用此程序。 */
将一假i置于栈内, 上
盖以状态3;
给出错误信息: 缺少
运算量。

状态	ACTION						GOTO
	i	+	*	(	)	#	E
0	s3	e1	e1	s2	e2	e1	1
1	e3	s4	s5	e3	e2	acc	
2	s3	e1	e1	s2	e2	e1	6
3	r4	r4	r4	r4	r4	r4	
4	s3	e1	e1	s2	e2	e1	7
5	s3	e1	e1	s2	e2	e1	8
6	e3	s4	s5	e3	s9	e4	
7	r1	r1	s5	r1	r1	r1	
8	r2	r2	r2	r2	r2	r2	
9	r3	r3	r3	r3	r3	r3	

5.3.7 LR分析中的出错处理

表5.9 文法5.11的LR(1)分析表
(含出错处理子程序)

e2: /* 处在状态0,2,4,5
时, 要求输入符号为
运算量的首符, 如i或
左括号。
处在状态1时, 要求输
入符号为运算符。
当遇到右括号时, 调
用此程序。 */
将下一输入符号(右
括号)删除;
给出错误信息: 右括
号不匹配。

状态	ACTION						GOTO
	i	+	*	(	)	#	E
0	s3	e1	e1	s2	e2	e1	1
1	e3	s4	s5	e3	e2	acc	
2	s3	e1	e1	s2	e2	e1	6
3	r4	r4	r4	r4	r4	r4	
4	s3	e1	e1	s2	e2	e1	7
5	s3	e1	e1	s2	e2	e1	8
6	e3	s4	s5	e3	s9	e4	
7	r1	r1	s5	r1	r1	r1	
8	r2	r2	r2	r2	r2	r2	
9	r3	r3	r3	r3	r3	r3	

5.3.7 LR分析中的出错处理

表5.9 文法5.11的LR(1)分析表
(含出错处理子程序)

e3: /* 处在状态1,6时, 要求输入符号为运算符, 当遇到i或左括号时, 调用此程序。 */
将 + 纳入栈顶, 上盖以状态4;
给出错误信息: 缺少运算符。

状态	ACTION						GOTO
	i	+	*	(	)	#	E
0	s3	e1	e1	s2	e2	e1	1
1	e3	s4	s5	e3	e2	acc	
2	s3	e1	e1	s2	e2	e1	6
3	r4	r4	r4	r4	r4	r4	
4	s3	e1	e1	s2	e2	e1	7
5	s3	e1	e1	s2	e2	e1	8
6	e3	s4	s5	e3	s9	e4	
7	r1	r1	s5	r1	r1	r1	
8	r2	r2	r2	r2	r2	r2	
9	r3	r3	r3	r3	r3	r3	

5.3.7 LR分析中的出错处理

表5.9 文法5.11的LR(1)分析表
(含出错处理子程序)

e4: /* 处在状态6时，要求输入符号为运算符或右括号，当遇到#时，调用此程序。*/
将)纳入栈顶，上盖以状态9；
给出错误信息：缺少右括号。

状态	ACTION						GOTO
	i	+	*	(	)	#	E
0	s3	e1	e1	s2	e2	e1	1
1	e3	s4	s5	e3	e2	acc	
2	s3	e1	e1	s2	e2	e1	6
3	r4	r4	r4	r4	r4	r4	
4	s3	e1	e1	s2	e2	e1	7
5	s3	e1	e1	s2	e2	e1	8
6	e3	s4	s5	e3	s9	e4	
7	r1	r1	s5	r1	r1	r1	
8	r2	r2	r2	r2	r2	r2	
9	r3	r3	r3	r3	r3	r3	

5.3.7 LR分析中的出错处理

文法(5.11) $G(E)$:

(1)

$E \rightarrow E + E$

(2) $E \rightarrow E * E$

(3) $E \rightarrow (E)$

(4) $E \rightarrow i$

表5.9 文法5.11的LR(1)分析表
(含出错处理子程序)

状态	ACTION						GOTO
	i	+	*	(	)	#	E
0	s3	e1	e1	s2	e2	e1	1
1	e3	s4	s5	e3	e2	acc	
2	s3	e1	e1	s2	e2	e1	6
3	r4	r4	r4	r4	r4	r4	
4	s3	e1	e1	s2	e2	e1	7
5	s3	e1	e1	s2	e2	e1	8
6	e3	s4	s5	e3	s9	e4	
7	r1	r1	s5	r1	r1	r1	
8	r2	r2	r2	r2	r2	r2	
9	r3	r3	r3	r3	r3	r3	

5.3.7 LR分析中的出错处理

输入符号串为: i+)

步骤	状态栈	符号栈	输入串	
1	0	#	i+)#	s3
2	03	#i	+)#	r4
3	01	#E	+)#	s4
4	014	#E+	)#	e2: 删除右括号
5	014	#E+	#	e1: 假i和3入栈
6	0143	#E+i	#	r4
7	0147	#E+E	#	r1
8	01	#E	#	acc

第五章

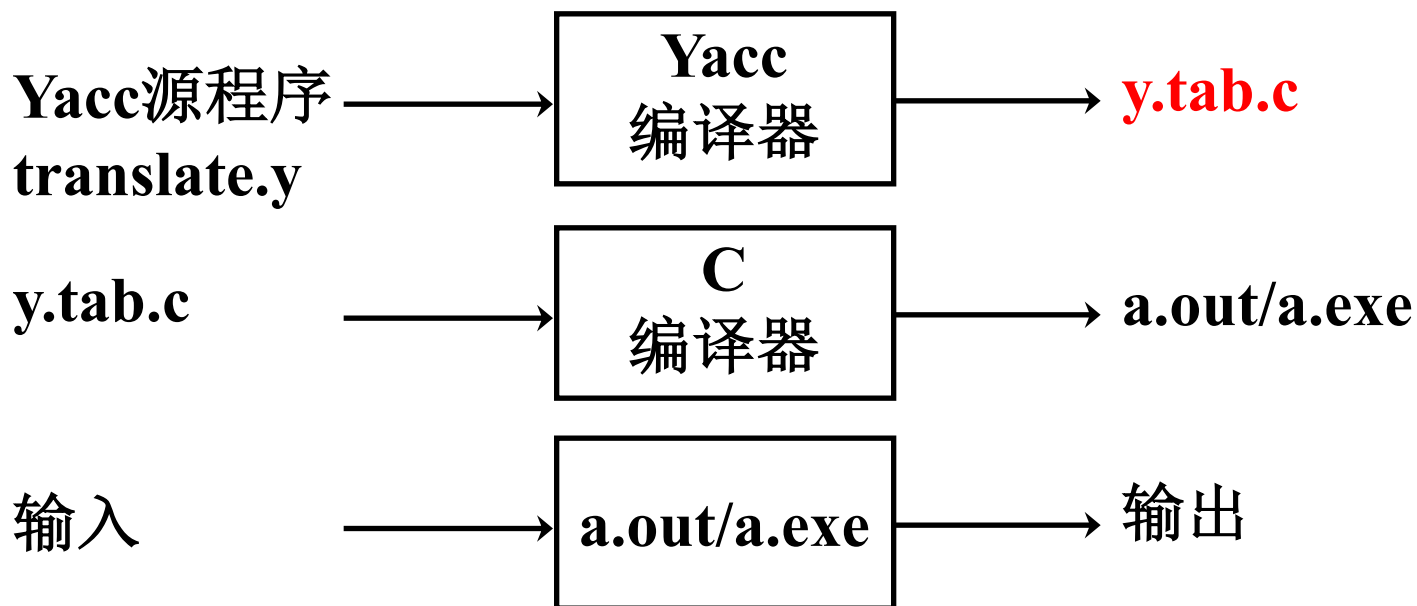
语法分析-自下而上分析

- 5.1 自下而上分析基本问题
- 5.2 算符优先分析
- 5.3 LR分析法
- 5.4 语法分析器的自动产生工具YACC

5.4 语法分析器的自动产生工具YACC

- **YACC**输入用户提供的语言的语法描述说明，基于**LALR**语法分析的原理，自动构造一个该语言的**语法分析器**，同时，它还能根据规格说明中给出的语义子程序建立规定的翻译。

5.4 语法分析器的自动产生工具YACC



- YACC源程序的结构

声明部分

%%

翻译规则

%%

辅助函数

```

%{
#include<stdio.h>
#include<ctype.h>
int yylex();
void yyerror (char const *);
%}

%token DIGIT

%%

line      :   expr '\n'           { printf("%d\n", $1); }
          ;

expr      :   expr '+' term       {$$ = $1 + $3; }
          |   term
          ;

term      :   term '*' factor     {$$ = $1 * $3; }
          |   factor
          ;

factor    :   '(' expr ')'        {$$ = $2; }
          |   DIGIT
          ;

%%

int main(){
    return yyparse();
}

int yylex(){
    int c;
    c = getchar();
    if(isdigit(c)){
        yylval = c - '0';
        return DIGIT;
    }
    return c;
}

void yyerror (char const *s){
    printf(stderr, "%s\n", s);
}

```